

Министерство образования Республики Беларусь

Учреждение образования  
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ  
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: Системное программирование (СП)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА  
к курсовому проекту  
на тему

**ПРОГРАММНОЕ СРЕДСТВО**  
**«Файловый менеджер с возможностью просмотра файлов на**  
**устройствах в одной локальной сети»**

БГУИР КП 1-40 01 01 101 ПЗ

Студент:

Бетень К.С.

Руководитель:

Деменковец Д.В.

Минск 2025

Учреждение образования  
«Белорусский государственный университет информатики  
и радиоэлектроники»

Факультет компьютерных систем и сетей

УТВЕРЖДАЮ  
Заведующий кафедрой

\_\_\_\_\_  
(подпись)

\_\_\_\_\_  
20\_\_ г.

ЗАДАНИЕ  
по курсовому проектированию

Студенту Бетене Константину Сергеевичу

1. Тема проекта Программное средство “Файловый менеджер с  
возможностью просмотра файлов на устройствах одной локальной сети”

2. Срок сдачи студентом законченного проекта 22 декабря 2025 г.

3. Исходные данные к проекту интегрированная среда разработки Clion, среда  
minGW на базе GCC компилятора, система контроля версий git, платформа для  
хостинга GitHub.

4. Содержание расчетно-пояснительной записки (перечень вопросов, которые  
подлежат разработке) Введение

1. Анализ предметной области

2. Проектирование программного средства

3. Разработка программного средства

4. Тестирование программного средства

5. Руководство пользователя

6. Список использованных источников

5. Перечень графического материала (с точным обозначением обязательных  
чертежей и графиков)

Чертеж А1 по ГОСТ 19.701–90

6. Консультант по проекту (с обозначением разделов проекта) Д.В. Деменковец

7. Дата выдачи задания 18 сентября 2025 г.

8. Календарный график работы над проектом на весь период проектирования (с обозначением сроков выполнения и трудоемкости отдельных этапов):

разделы 1,2 к 25.09 – 15 %;

раздел 3 к 13.10 – 10 %;

разделы 4,5 к 27.10 – 10 %;

разделы 6,7 к 10.11 – 35 %;

раздел 8 к 17.11 – 5 %;

раздел 9 к 22.11 – 10 %;

оформление пояснительной записки и графического материала к 01.12 – 15 %

Защита курсового проекта с 02.12 по 22.12

РУКОВОДИТЕЛЬ Д.В. Деменковец

(подпись)

Задание принял к исполнению К.С. Бетенья

(дата и подпись студента)

## СОДЕРЖАНИЕ

|                                                                    |    |
|--------------------------------------------------------------------|----|
| Введение .....                                                     | 5  |
| 1 Анализ предметной области .....                                  | 6  |
| 1.1 Аналоги программного средства .....                            | 6  |
| 1.2 Постановка задачи .....                                        | 9  |
| 2 Разработка структурной схемы .....                               | 10 |
| 2.1 Структура программы .....                                      | 10 |
| 2.2 Интерфейс программного средства .....                          | 10 |
| 2.3 Проектирование функциональных требования .....                 | 12 |
| 3 Разработка программного средства .....                           | 18 |
| 3.1 Работа программного средства .....                             | 18 |
| 3.2 Работа с сетевыми устройствами .....                           | 21 |
| 3.3 Изменение настроек .....                                       | 25 |
| 4 Тестирование программного средства .....                         | 28 |
| 4.1 Ошибка компиляции из-за отсутствующего заголовочного файла ... | 28 |
| 4.2 Ошибка сборки из-за неправильных флагов компилятора .....      | 29 |
| 5 Руководство пользователя .....                                   | 30 |
| 5.1 Интерфейс программного средства .....                          | 30 |
| 5.2 Управление программным средством .....                         | 32 |
| Заключение .....                                                   | 36 |
| Список использованных источников .....                             | 37 |
| Приложение А (обязательное) Исходный код .....                     | 38 |

## ВВЕДЕНИЕ

В условиях повсеместной цифровизации и роста объемов данных, эффективное управление файлами и обеспечение к ним беспрепятственного доступа становятся критически важными задачами как для корпоративной среды, так и для частных пользователей. Современные рабочие процессы часто предполагают взаимодействие нескольких устройств — персональных компьютеров, ноутбуков, планшетов и смартфонов, — что создает потребность в унифицированном и удобном доступе к распределенным информационным ресурсам. В этом контексте особую актуальность приобретают решения, позволяющие централизованно управлять файлами, расположенными на различных устройствах в пределах одной локальной сети.

Файловый менеджер с сетевыми возможностями — это специализированное программное обеспечение, которое не только предоставляет стандартный инструментарий для работы с файловой системой локального устройства, но и интегрирует функции для просмотра и управления файлами на других компьютерах и носителях информации в рамках единой сети. Это способствует оптимизации коллективной работы, упрощает обмен данными и устраняет необходимость использования физических носителей или сторонних облачных сервисов для внутреннего обмена.

Основной задачей подобного файлового менеджера является обеспечение безопасного, стабильного и интуитивно понятного доступа к сетевым ресурсам. Это достигается за счет поддержки технологии универсального plug-and-play обнаружения устройств, а также разграничения прав доступа. Таким образом, пользователь получает возможность просматривать фотографии, документы, медиафайлы и другие данные, хранящиеся на другом устройстве в сети, с тем же уровнем удобства, как если бы они находились на его локальном диске.

Развитие и внедрение таких решений оказывает значительное влияние на организацию цифрового пространства. Они способствуют снижению операционных издержек, связанных с передачей информации, повышают мобильность и гибкость рабочих процессов, а также усиливают контроль над распределенными данными.

Таким образом, тема разработки и использования файловых менеджеров с возможностью просмотра файлов в локальной сети является высокоактуальной в контексте развития концепций цифрового офиса и повседневной многодевайсности. Изучение принципов их работы, функциональных возможностей и архитектуры позволяет глубже понять тенденции в области управления децентрализованными данными и сформировать представление о современных подходах к созданию единой цифровой экосистемы.

# 1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

## 1.1 Аналоги программного средства

### 1.1.1 Системный проводник «Проводник Windows»

Стандартный файловый менеджер операционной системы Microsoft Windows, интегрированный в её среду, является наиболее распространённым базовым инструментом для управления файлами на локальных дисках. Однако его функциональность не ограничивается локальным компьютером. «Проводник Windows» предоставляет встроенные возможности для работы в локальной сети через поддержку протокола SMB (Server Message Block).

Интерфейс проводника знаком абсолютному большинству пользователей Windows, что минимизирует порог вхождения. Он обеспечивает базовые операции: навигацию, копирование, перемещение, удаление и поиск файлов как на локальных, так и на сетевых ресурсах. Визуализация структуры папок и предпросмотр файлов (особенно изображений) делают его удобным для повседневного использования.

Рассмотреть файловый менеджер можно на рисунке 1.1.

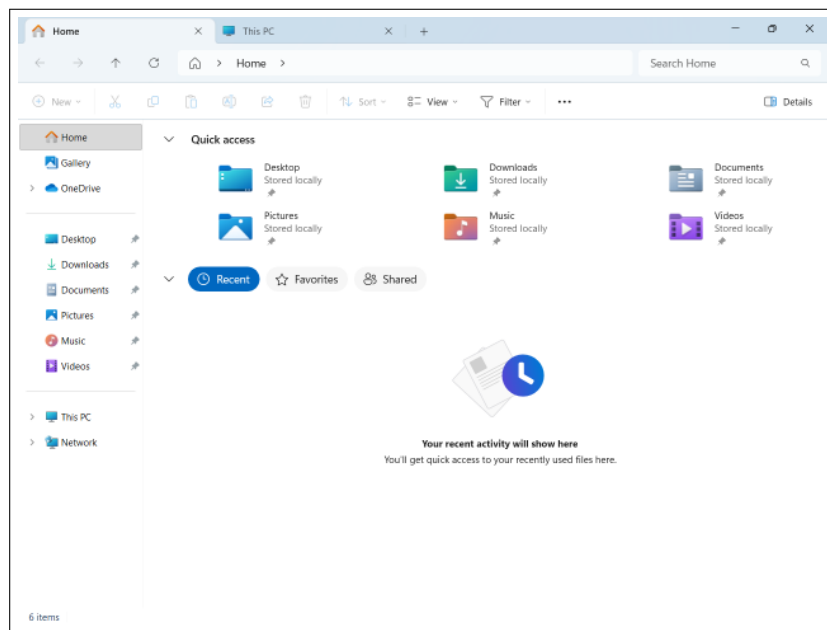


Рисунок 1.1 – Программное средство «Проводник Windows»

К преимуществам «Проводник Windows» как сетевого клиента можно отнести его полную бесплатность и немедленную доступность, тесную интеграцию с операционной системой и службами Windows, а также стабильность работы с ресурсами в доменных сетях. Главные недостатки — это ограниченная функциональность для продвинутой работы (например, отсутствие двухпанельного режима), слабые возможности фильтрации и сортировки файлов на сетевых ресурсах, а также часто нестабильная работа функции автоматического обнаружения устройств в разделе «Сеть».

в современных версиях ОС, что вынуждает пользователей вручную прописывать сетевые пути.

### 1.1.2 Проводник Mac OS «Finder»

Нативный файловый менеджер операционной системы macOS, выполняющий аналогичные «Проводник Windows» функции, но с глубокой интеграцией в экосистему Apple. «Finder» является основным инструментом для организации файлов, поиска и работы с внешними накопителями. Его сетевая функциональность реализована через поддержку протоколов SMB, AFP (Apple Filing Protocol) и FTP.

Особенностью «Finder» является бесшовный доступ к сетевым ресурсам через боковую панель в разделе «Сети», где автоматически отображаются доступные компьютеры в локальной сети. После подключения сетевая папка может быть смонтирована как локальный диск, обеспечивая прозрачную работу с файлами. Интерфейс выдержан в характерном для macOS минималистичном и интуитивном стиле, с поддержкой тегов, быстрого предпросмотра (Quick Look) и режимов отображения (иконки, список, колонки).

Интерфейс программного средства представлен на рисунке 1.2.

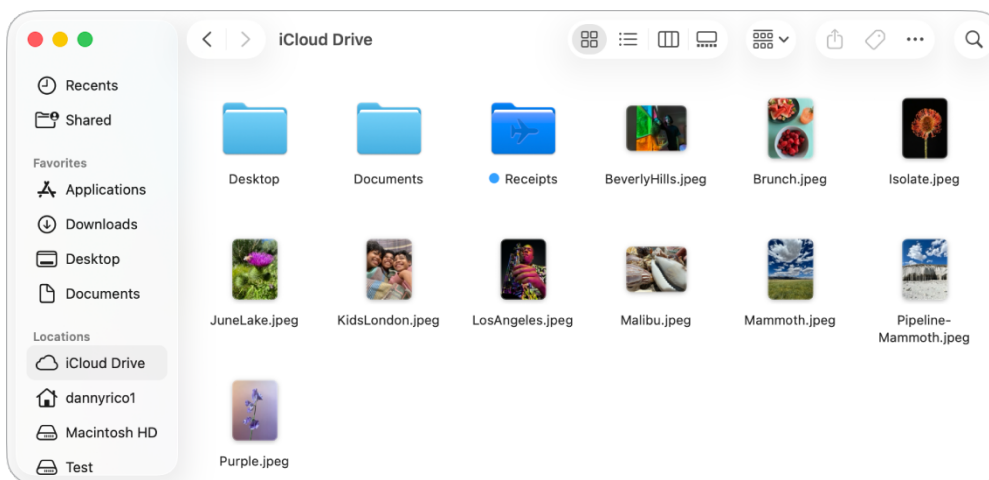


Рисунок 1.2 – Программное средство «Finder»

К преимуществам «Finder» относятся его стабильная и быстрая работа в сетях Apple (с использованием Bonjour для автоматического обнаружения), удобный механизм предпросмотра файлов и высокая степень интеграции с другими приложениями macOS. К недостаткам можно отнести ограниченные возможности по кастомизации интерфейса, менее гибкое управление сетевыми подключениями по сравнению с профессиональными менеджерами и ориентацию в первую очередь на экосистему Apple, что может создавать сложности при работе в гетерогенных сетях с преобладанием устройств на Windows.

### 1.1.3 Менеджер «Total Commander»

Мощный двухпанельный файловый менеджер для операционной системы Windows, считающийся профессиональным стандартом для работы с файлами. В отличие от нативных проводников, «Total Commander» изначально создан для эффективного управления большими объёмами данных, в том числе расположенными в сети. Его сетевая функциональность является одной из ключевых и реализуется как через встроенную поддержку протоколов (FTP, FTPS, WebDAV), так и через прямое подключение к сетевым папкам Windows (SMB) и плагины для доступа к облачным хранилищам.

Интерфейс «Total Commander» построен по классической двухпанельной схеме (Norton Commander), что идеально подходит для синхронизации, сравнения каталогов и массовых операций с файлами между локальной и сетевой панелями. Программа обладает огромным набором функций: расширенный поиск, групповое переименование, работа с архивами как с папками, сравнение файлов и многое другое.

Рассмотреть данный сервис можно на рисунке 1.3.

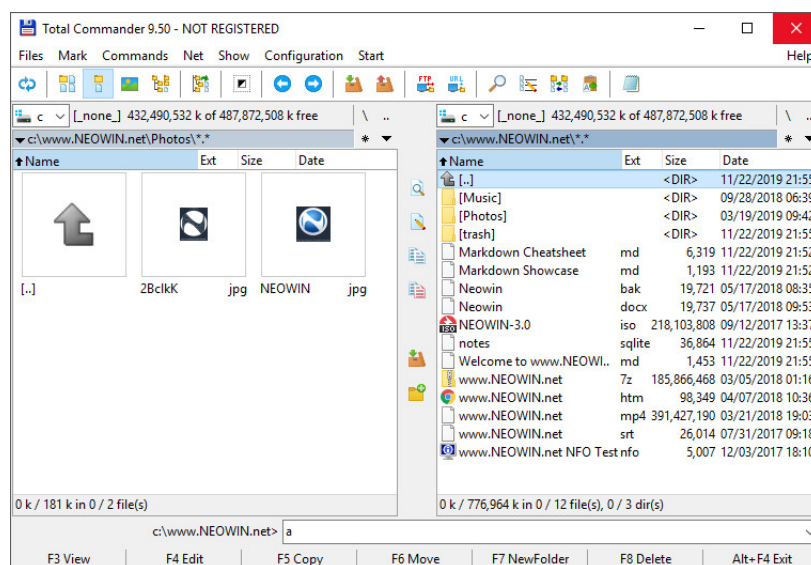


Рисунок 1.3 – Программное средство «Total Commander»

К неоспоримым преимуществам «Total Commander» для сетевой работы относятся его высочайшая функциональность и гибкость, поддержка множества протоколов из «коробки», высокая скорость операций за счёт оптимизированных алгоритмов и возможность тонкой настройки под конкретные задачи. Основные недостатки — это платная лицензионная модель (с длительным пробным периодом), архаичный для нового пользователя интерфейс, требующий времени на освоение, и отсутствие нативных версий для macOS и Linux, что ограничивает его использование в кросс-платформенных средах.



## 1.2 Постановка задачи

Цель курсовой работы – проектирование и реализация файлового менеджера с возможностью просмотра файлов на устройствах в одной локальной сети в виде кроссплатформенного настольного приложения.

- модуль утилит;
- модуль настроек;
- модуль файловой системы;
- модуль сетевых операций;
- модуль интерфейса пользователя;

При разработке программного средства будет использована интегрированная среда разработки CLion [1] с использованием языка программирования C++ [2] и платформы MinGW [3] для сборки проекта.

Проект будет разработан с использованием модульной архитектуры [4], где каждый модуль будет отвечать за свою функциональность:

- Разделение ответственности – каждый модуль имеет чётко определённую область ответственности;
- Инкапсуляция – модули скрывают детали реализации и предоставляют интерфейсы для взаимодействия;
- Масштабируемость – модульная структура позволяет легко добавлять новую функциональность;
- Поддерживаемость – изолированные модули упрощают отладку и модификацию кода;

## **2 РАЗРАБОТКА СТРУКТУРНОЙ СХЕМЫ**

### **2.1 Структура программы**

Было принято решение, что при разработке данного программного средства будут использоваться пять основных модулей и девять подмодулей:

- Utils – вспомогательные функции для работы с путями и форматированием данных;
- Settings – управление настройками приложения, темами, шрифтами и их сохранение в реестре Windows;
- FileSystem – операции с файловой системой (навигация по директориям, работа с деревом каталогов, получение информации о файлах);
- Network – сканирование сетевых устройств, перечисление сетевых ресурсов, работа с сетевыми дисками и USB устройствами;
- UI – графический интерфейс приложения, включающий следующие подмодули:
  - List – управление отображением файлов в различных режимах (подробно, крупные значки, мелкие значки);
  - Layout – размещение элементов интерфейса и адаптация к изменению размера окна;
  - Navigation – навигация по файловой системе, управление историей переходов, обновление состояния интерфейса;
  - Menu – создание и управление меню приложения, обработка горячих клавиш;
  - Theme – применение тем оформления (светлая, тёмная, системная) и настройка шрифтов;
  - SubClass – отрисовка элементов управления для поддержки тёмной темы;
  - Window – хранение и управление состоянием окна, инкапсуляция доступа к данным;
  - Control – создание и настройка всех элементов интерфейса;
  - WindowProc – обработка всех сообщений Windows и координация работы модулей;

Каждый модуль будет содержать интерфейсный файл для доступа к функциональности модуля из других модулей.

### **2.2 Интерфейс программного средства**

Для проектирования интерфейса программного средства было принято решение использовать нативный Win32 API [5], который предоставляет стандартные компоненты операционной системы Windows. Данный подход обеспечивает нативный внешний вид приложения, соответствующий

системным стандартам, а также высокую производительность и минимальное потребление ресурсов.

Для проектирования архитектуры интерфейса было принято решение использовать модульную структуру, где каждый компонент интерфейса инкапсулирован в отдельном модуле, что обеспечивает разделение ответственности и упрощает поддержку кода.

### **2.2.1 Блочные компоненты**

Блочные компоненты представляют собой основные элементы интерфейса, которые используются для построения окна приложения.

В данном программном средстве используются следующие блочные компоненты:

- TreeView – иерархическое дерево для отображения структуры каталогов и сетевых устройств;
- ListView – список для отображения файлов и папок в различных режимах отображения (подробно, крупные значки, мелкие значки);
- StatusBar – строка состояния для отображения информации о количестве файлов и папок в текущей директории;
- Splitter – разделитель между верхним и нижним деревом каталогов для изменения размера областей;

Блочные компоненты могут содержать в себе другие компоненты, а также стили и логику, которые необходимы для их работы. Для кастомной отрисовки компонентов в тёмной теме используется механизм подклассирования окон (window subclassing).

### **2.2.2 Компоненты управления**

Компоненты управления представляют собой элементы интерфейса, которые позволяют пользователю взаимодействовать с приложением.

В данном программном средстве используются следующие компоненты управления:

- Button – кнопки навигации (назад, вперёд) и переключения режимов отображения (подробно, крупные значки, мелкие значки);
- Edit – поле ввода для указания пути к директории с возможностью перехода по нажатию Enter или кнопки "Перейти";
- Menu – главное меню приложения с подменю для настройки темы, шрифта и размера текста;
- Accelerator – горячие клавиши для быстрого доступа к функциям приложения;

Для работы с навигацией используется механизм истории переходов, который позволяет отслеживать последовательность посещённых директорий и обеспечивает функциональность кнопок "Назад" и "Вперёд". Для работы с меню используется стандартный Win32 API, который обеспечивает интеграцию с системным меню Windows.

### **2.2.3 Компоненты отображения**

Компоненты отображения представляют собой элементы интерфейса, которые отображают информацию пользователю.

В данном программном средстве используются следующие компоненты отображения:

- Static – для отображения текстовых меток и пустых сообщений, когда директория не содержит файлов;
- Icon – системные иконки файлов и папок, получаемые через Shell API для корректного отображения типов файлов;
- Tooltip – всплывающие подсказки для кнопок управления, отображающие описание функций и горячие клавиши;
- Header – заголовки колонок в режиме подробного отображения (Имя, Тип, Изменён);

Для работы с темами оформления используется механизм определения системной темы через Windows API, а также кастомная отрисовка элементов управления для поддержки тёмной темы. Для работы с шрифтами используется механизм динамического изменения размера шрифта с автоматической адаптацией размеров элементов интерфейса.

## **2.3 Проектирование функциональных требования**

Функционал программного средства – ключевая точка разработки программного средства, составляющей которого являются алгоритмы. В связи с этим, программное средство «Файловый менеджер с возможностью просмотра файлов на устройствах в одной локальной сети» должно обладать некоторыми алгоритмами:

- алгоритм загрузки настроек приложения из реестра;
- алгоритм расширения узла дерева каталога;
- алгоритм расширения сетевого узла;
- алгоритм обновления строки состояния директории;
- алгоритм обработки файлов директории;

### **2.3.1 Алгоритм загрузки настроек приложения**

Система управления настройками позволяет сохранять информацию о пользовательских предпочтениях (тема оформления, размер и тип шрифта) в реестре Windows и предоставляет возможность восстановления этих параметров при следующем запуске приложения.

Алгоритм загрузки настроек приложения из реестра представлен на рисунке 5.3.

Перед началом работы необходимо открыть ключ реестра, в котором хранятся настройки приложения. Далее, при успешном открытии ключа, последовательно считываются три параметра: тема оформления, размер шрифта и имя шрифта. Для каждого параметра выполняется проверка

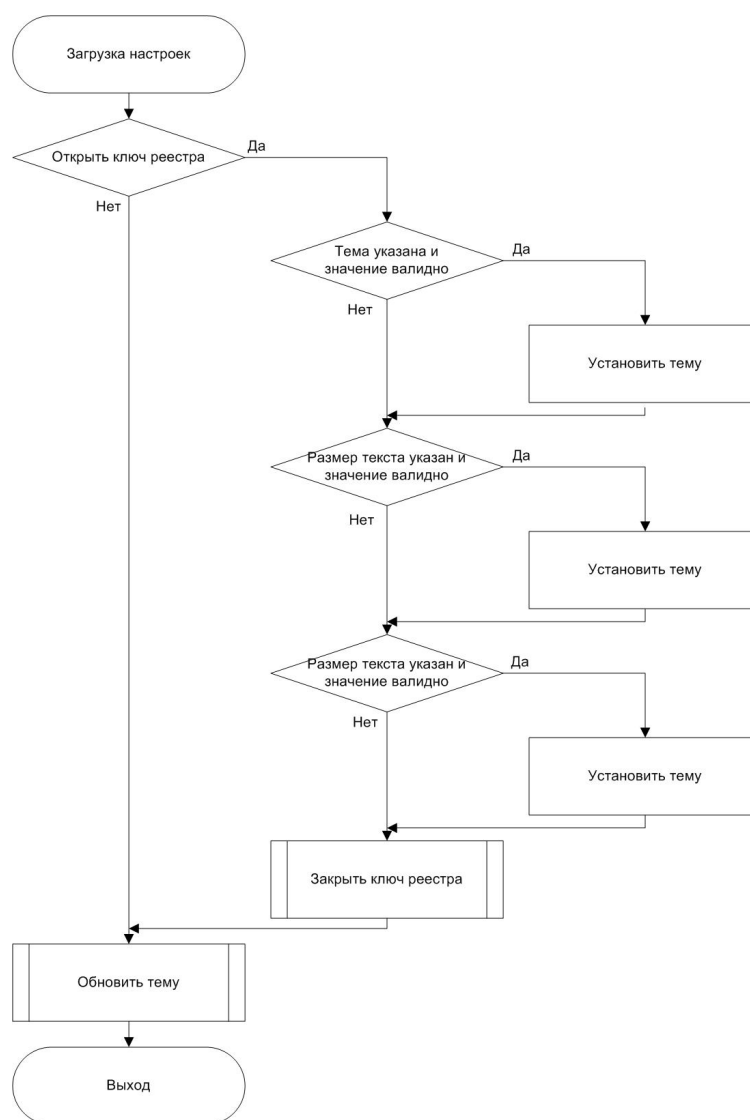


Рисунок 2.1 – Алгоритм загрузки настроек приложения из реестра

успешности чтения и валидация значения. Если параметр успешно прочитан и валиден, он применяется к текущей конфигурации. После обработки всех параметров ключ реестра закрывается, и вызывается функция обновления цветов темы для применения загруженных настроек к интерфейсу приложения.

### 2.3.2 Алгоритм расширения узла дерева каталога

Система навигации по файловой системе – это система, которая позволяет отображать структуру каталогов в виде дерева и обеспечивает динамическую загрузку подкаталогов.

Система навигации по файловой системе позволяет отображать иерархическую структуру директорий, обеспечивает ленивую загрузку содержимого и предоставляет возможность эффективной навигации по большим файловым системам.

Алгоритм расширения узла дерева каталога представлен на рисунке 2.2.

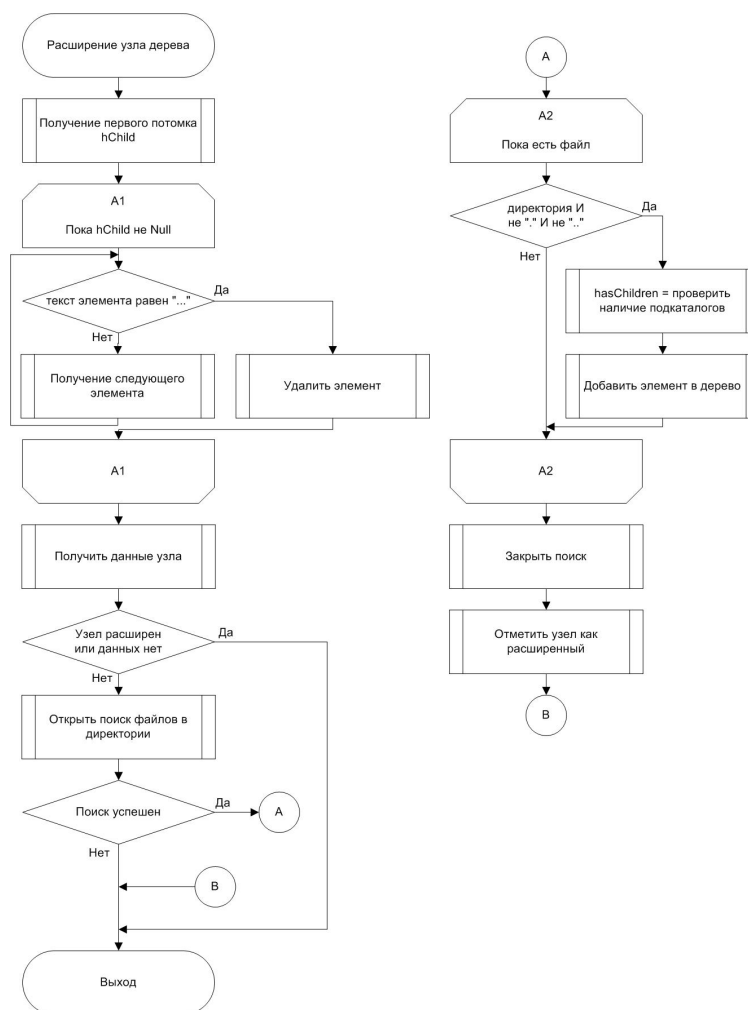


Рисунок 2.2 – Алгоритм расширения узла дерева каталога

Перед началом работы необходимо удалить все placeholder-элементы, которые были добавлены для обозначения возможности расширения узла. Далее проверяется, был ли узел уже расширен ранее, чтобы избежать повторной загрузки. Если узел ещё не расширен, открывается поиск файлов в соответствующей директории. При успешном открытии поиска выполняется перебор всех найденных элементов: пропускаются системные директории, для каждой найденной директории проверяется наличие подкаталогов, и элемент добавляется в дерево с соответствующей пометкой о наличии дочерних элементов. После завершения перебора поиск закрывается, и узел помечается как расширенный.

### 2.3.3 Алгоритм расширения сетевого узла

Система работы с сетевыми ресурсами – это система, которая позволяет обнаруживать и отображать сетевые устройства и общие ресурсы в локальной сети.

Система работы с сетевыми ресурсами позволяет находить сетевые серверы и шары, обеспечивает динамическую загрузку списка доступных ресурсов и предоставляет возможность навигации по сетевым директориям.

Алгоритм расширения сетевого узла представлен на рисунке 2.3.

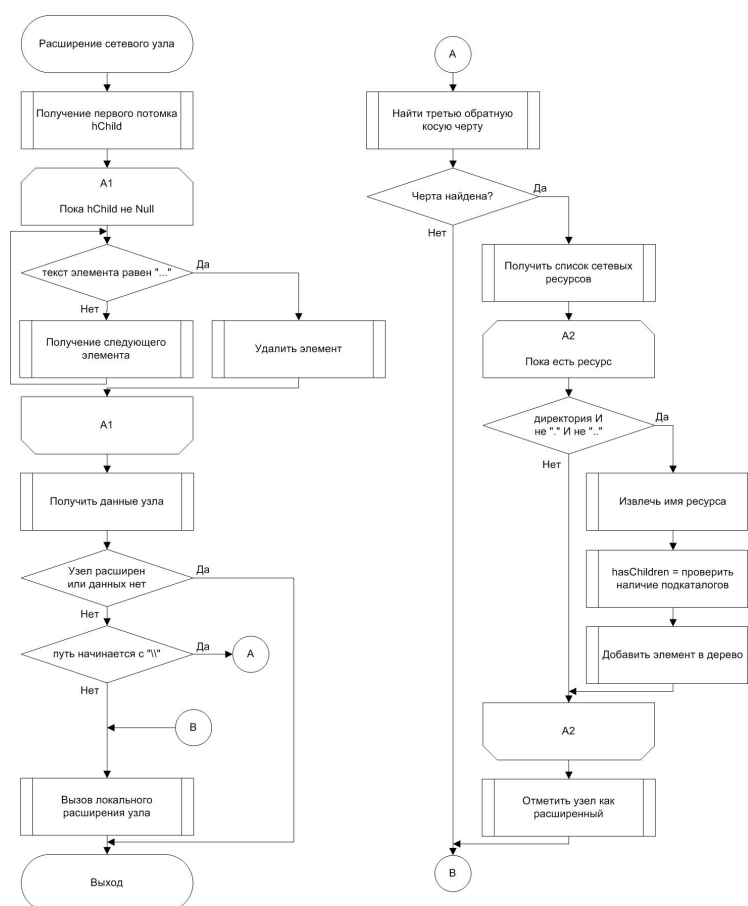


Рисунок 2.3 – Алгоритм расширения сетевого узла

Перед началом работы необходимо удалить все placeholder-элементы из узла сетевого устройства. Далее проверяется, был ли узел уже расширен, чтобы избежать повторной загрузки сетевых ресурсов. Если узел ещё не расширен, определяется тип сетевого пути: проверяется наличие третьей обратной косой черты в пути. Если третья черта не найдена, это означает, что путь указывает на сетевой сервер, и выполняется перебор всех доступных сетевых ресурсов на этом сервере. Для каждого найденного ресурса извлекается его имя, проверяется наличие подкаталогов, и ресурс добавляется в дерево. Если третья черта найдена, это означает, что путь указывает на конкретную шару или подкаталог, и для его обработки вызывается стандартный алгоритм расширения локального узла дерева.

#### 2.3.4 Алгоритм обновления строки состояния директории

Система отображения статистики директории – это система, которая позволяет отображать информацию о количестве файлов и папок в текущей директории.

Система отображения статистики директории позволяет подсчитывать количество элементов различных типов, суммировать размеры файлов и предоставляет возможность отображения этой информации пользователю в строке состояния интерфейса.

Алгоритм обновления строки состояния директории представлен на рисунке 2.4.

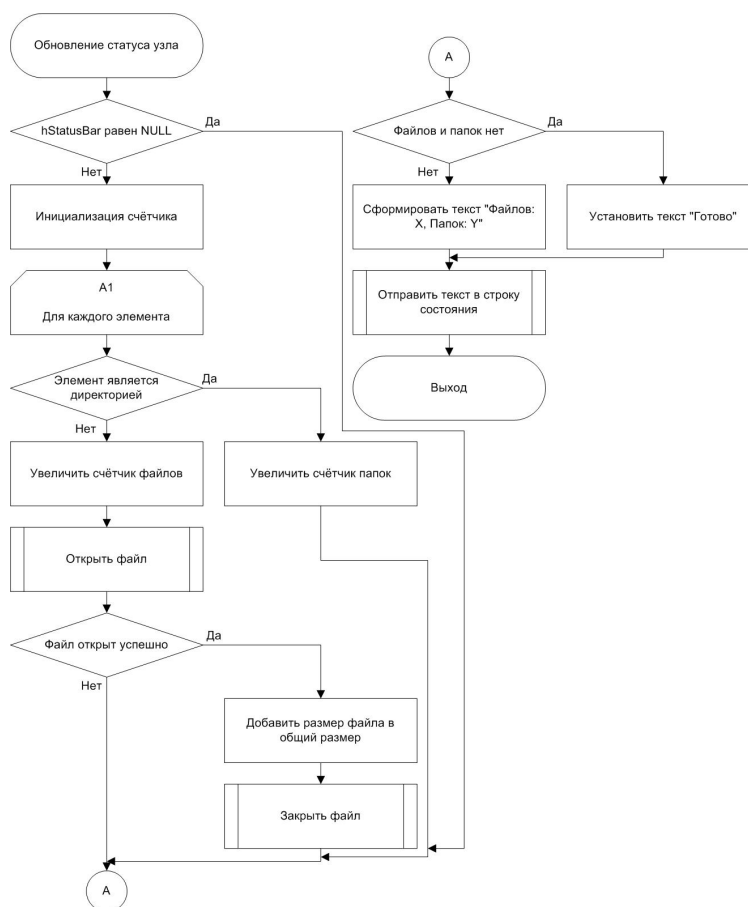


Рисунок 2.4 – Алгоритм обновления строки состояния директории

Перед началом работы необходимо проверить существование окна строки состояния. Далее инициализируются счётчики файлов и папок. Выполняется перебор всех элементов в текущей директории: для каждого элемента определяется его тип. Если элемент является директорией, увеличивается счётчик папок. Если элемент является файлом, увеличивается счётчик файлов, и выполняется попытка открыть файл для получения его размера. При успешном открытии файла его размер добавляется к общему размеру, после чего файл закрывается. После обработки всех элементов проверяется, есть ли в директории файлы и папки: если директория пуста, в строку состояния устанавливается текст "Готово", иначе формируется текст с количеством файлов и папок. Сформированный текст отправляется в окно строки состояния для отображения пользователю.

### 2.3.5 Алгоритм обработки файлов директории

Система отображения содержимого директории – это система, которая позволяет получать список файлов и папок в директории и отображать их в интерфейсе приложения.



Система отображения содержимого директории позволяет перебирать элементы файловой системы, определять их типы, получать системные иконки и предоставляет возможность отображения элементов в виде списка с информацией о типе и времени последнего изменения.

Алгоритм обработки файлов директории представлен на рисунке 2.5.

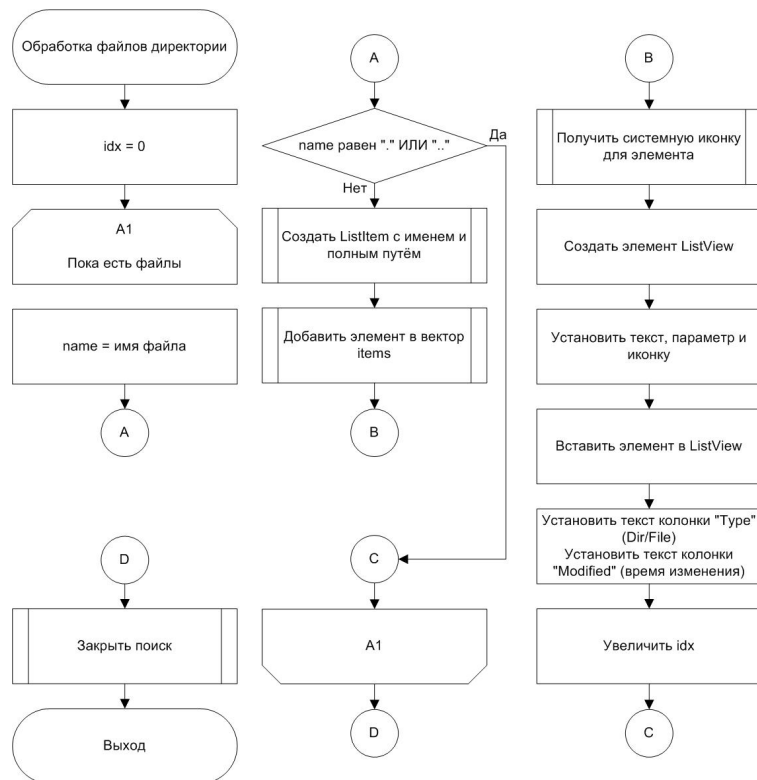


Рисунок 2.5 – Алгоритм обработки файлов директории

Перед началом работы инициализируется счётчик элементов. Выполняется перебор всех найденных файлов и директорий через функцию поиска файловой системы: для каждого найденного элемента проверяется его имя. Если имя элемента равно "." или "..", элемент пропускается. Для остальных элементов определяется тип, создаётся объект элемента списка с именем и полным путём, и элемент добавляется в вектор элементов. Далее для каждого элемента получается системная иконка, соответствующая его типу, создаётся элемент списка интерфейса с установкой текста, параметра и иконки. Элемент вставляется в список интерфейса, после чего устанавливается текст колонки "Type" и текст колонки "Modified" с информацией о времени последнего изменения. Счётчик элементов увеличивается, и обработка продолжается до тех пор, пока есть следующие файлы. После завершения перебора поиск файлов закрывается.

## 3 РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

При разработке программного средства будет использована интегрированная среда разработки CLion с использованием языка программирования C++ и платформы MinGW для сборки проекта. С целью улучшения поддержки исходного кода в ходе разработки активно использовался комплект библиотек, предлагаемый выбранной языковой платформой.

### 3.1 Работа программного средства

К основному функционалу программного средства можно отнести следующие компоненты:

- получение списка директорий;
- получение списка файлов в директории;
- открытие файла;

#### 3.1.1 Получение списка директорий

Получение списка директорий начинается с проверки наличия placeholder-элементов в узле дерева каталогов и их удаления.

После удаления placeholder-элементов функция проверяет данные узла на существование и проверяет, был ли узел уже расширен ранее, чтобы избежать повторной загрузки подкаталогов.

Если узел ещё не расширен, формируется паттерн поиска для директории и открывается поиск файлов через функцию FindFirstFileW. При успешном открытии поиска выполняется перебор всех найденных элементов: пропускаются файлы, пропускаются системные директории, для каждой найденной директории проверяется наличие подкаталогов и элемент добавляется в дерево с соответствующей пометкой о наличии дочерних элементов.

```
void ExpandTreeItem(HWND hTree, HTREEITEM hItem) {
    // Remove placeholder children
    HTREEITEM hChild = TreeView_GetChild(hTree, hItem);
    while (hChild) {
        TVITEMW tvi{}; tvi.mask = TVIF_TEXT; tvi.hItem = hChild;
        wchar_t buf[8]; tvi.pszText = buf; tvi.cchTextMax = 8;
        TreeView_GetItem(hTree, &tvi);
        if (wcscmp(buf, L"...") == 0) {
            TreeView_DeleteItem(hTree, hChild);
            break;
        }
        hChild = TreeView_GetNextSibling(hTree, hChild);
    }

    TVITEMW tvi{}; tvi.mask = TVIF_PARAM; tvi.hItem = hItem;
    TreeView_GetItem(hTree, &tvi);
    NodeData *data = reinterpret_cast<NodeData*>(tvi.lParam);
    if (!data || data->expanded) return;
```

```

std::wstring pattern = Utils::JoinPath(data->path, L"*");
WIN32_FIND_DATA fd{};
HANDLE h = FindFirstFileW(pattern.c_str(), &fd);
if (h == INVALID_HANDLE_VALUE) return;
do {
    if ((fd.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY) == 0)
        continue;
    std::wstring name = fd.cFileName;
    if (name == L"." || name == L"..") continue;
    std::wstring full = Utils::JoinPath(data->path, name);
    bool hasChildren = HasSubdirs(full);
    InsertTreeItem(hTree, hItem, name, full, hasChildren);
} while (FindNextFileW(h, &fd));
FindClose(h);
data->expanded = true;
}

```

После завершения перебора поиск закрывается, и узел помечается как расширенный, чтобы предотвратить повторную загрузку при следующем раскрытии.

Если в процессе работы алгоритма произошла ошибка, функция завершается без добавления элементов, и узел остаётся в исходном состоянии.

### 3.1.2 Получение списка файлов в директории

Получение списка файлов в директории начинается с очистки списка интерфейса и вектора элементов. Далее выполняется нормализация пути директории для разрешения символических ссылок и коротких путей.

После нормализации пути проверяется существование директории через `GetFileAttributesW`. Если путь не существует, отображается соответствующее сообщение об ошибке, и функция завершается.

Если путь существует, проверяется, что это действительно директория, а не файл. Если переданный путь указывает на файл, отображается сообщение "Файл пуст", и функция завершается.

```

int idx = 0;
do {
    std::wstring name = fd.cFileName;
    if (name == L"." || name == L"..") continue;
    bool isDir = (fd.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY) != 0;

    ListItem ri{ name, Utils::JoinPath(normalizedPath, name), isDir };
    items.push_back(ri);

    LVITEMW item{};
    SHFILEINFOW sfi{};
    UINT attrs = isDir ? FILE_ATTRIBUTE_DIRECTORY :
        FILE_ATTRIBUTE_NORMAL;
    SHGetFileInfoW(ri.fullPath.c_str(), attrs, &sfi, sizeof(sfi),
        SHGFI_SYSICONINDEX | SHGFI_USEFILEATTRIBUTES |
        SHGFI_SMALLICON);
}

```

```

item.mask = LVIF_TEXT | LVIF_PARAM | LVIF_IMAGE;
item.iItem = idx;
item.pszText = const_cast<wchar_t*>(name.c_str());
item.lParam = static_cast<LPARAM>(idx);
item.iImage = sfi.iIcon;
ListView_InsertItem(hList, &item);

std::wstring typeText = isDir ? L"Dir" : L"File";
ListView_SetItemText(hList, idx, 1, const_cast<wchar_t*>(typeText.
    c_str()));

std::wstring mod = Utils::FileTimeToWString(fd.ftLastWriteTime);
ListView_SetItemText(hList, idx, 2, const_cast<wchar_t*>(mod.c_str()
    ));
++idx;
} while (FindNextFileW(h, &fd));
FindClose(h);

```

Для директории формируется паттерн поиска и открывается поиск файлов. При успешном открытии выполняется перебор всех найденных элементов: пропускаются системные директории, для каждого элемента определяется его тип, создаётся объект элемента списка, получается системная иконка, и элемент добавляется в список интерфейса с установкой текста колонок "Type" и "Modified".

После завершения перебора поиск закрывается, и список файлов отображается пользователю.

### 3.1.3 Открытие файла

Открытие файла начинается с обработки события двойного клика пользователя по элементу списка. Функция получает индекс выбранного элемента и проверяет его корректность.

После получения индекса функция извлекает соответствующий элемент из списка элементов директории и проверяет его тип. Если выбранный элемент является директорией, выполняется переход в эту директорию через функцию NavigateTo.

```

int idx = act->iItem;
if (idx >= 0 && idx < (int)state->listItems.size()) {
    const Filesystem::ListItem &ri = state->listItems[idx];
    if (ri.isDir) {
        NavigateTo(hwnd, *controls, *state, ri.fullPath, true);
    } else {
        HINSTANCE h = ShellExecuteW(hwnd, L"open", ri.fullPath.c_str(),
            nullptr, nullptr, SW_SHOWNORMAL);

        if ((INT_PTR)h <= 32) {
            MessageBoxW(hwnd, L"Не удалось открыть файл ", L"Ошибка",
                MB_ICONERROR | MB_OK);
        }
    }
}

```

Если выбранный элемент является файлом, выполняется попытка открыть файл через функцию ShellExecuteW с параметром "open",

который передаёт управление файлом операционной системе для выбора соответствующего приложения.

При успешном открытии файла функция ShellExecuteW возвращает значение больше 32. Если возвращаемое значение меньше или равно 32, это означает ошибку открытия файла, и пользователю отображается сообщение об ошибке "Не удалось открыть файл".

Если файл успешно открыт, операционная система запускает соответствующее приложение для работы с файлом, и пользователь может работать с содержимым файла.

## 3.2 Работа с сетевыми устройствами

К функционалу работы с сетевыми устройствами можно отнести следующие компоненты:

- поиск сетевых устройств;
- получение списка сетевых шаров;
- поиск файлов и шаров на сетевых устройствах;

### 3.2.1 Поиск сетевых устройств

Поиск сетевых устройств начинается с инициализации множества для хранения уникальных устройств и вектора для результата.

Алгоритм использует несколько методов для максимально полного обнаружения сетевых устройств. Первый метод – получение сетевых дисков, сопоставленных буквам (Z:, Y: и т.д.), которые добавляются в множество устройств.

```
std::vector<std::wstring> networkDrives = EnumerateNetworkDrives();  
for (const auto& dev : networkDrives) {  
    if (!dev.empty()) {  
        devicesSet.insert(dev);  
    }  
}
```

Второй метод – рекурсивное перечисление всех сетевых ресурсов, начиная с корня сети, что позволяет найти сетевые компьютеры и шары аналогично представлению "Сеть" в проводнике Windows.

```
NETRESOURCE* pNetResource = nullptr;  
EnumerateNetworkResourcesRecursive(pNetResource, devicesSet, 0);
```

Третий метод – перечисление рабочих групп и доменов с их серверами через установку типа отображения RESOURCEDISPLAYTYPE\_DOMAIN.

```
NETRESOURCE nr{};  
nr.dwScope = RESOURCE_GLOBALNET;  
nr.dwType = RESOURCETYPE_ANY;  
nr.dwUsage = RESOURCEUSAGE_CONTAINER;  
nr.dwDisplayType = RESOURCEDISPLAYTYPE_DOMAIN;  
EnumerateNetworkResourcesRecursive(&nr, devicesSet, 0);
```

Четвёртый метод – прямое перечисление серверов через установку типа отображения `RESOURCE_DISPLAYTYPE_SERVER`.

```
nr.dwScope = RESOURCE_GLOBALNET;
nr.dwType = RESOURCETYPE_ANY;
nr.dwUsage = RESOURCEUSAGE_ALL;
nr.dwDisplayType = RESOURCE_DISPLAYTYPE_SERVER;
EnumerateNetworkResourcesRecursive(&nr, devicesSet, 0);
```

Пятый метод – перечисление всех дисковых шаров через установку типа ресурса `RESOURCETYPE_DISK`.

```
nr.dwScope = RESOURCE_GLOBALNET;
nr.dwType = RESOURCETYPE_DISK;
nr.dwUsage = RESOURCEUSAGE_ALL;
nr.dwDisplayType = RESOURCE_DISPLAYTYPE_GENERIC;
EnumerateNetworkResourcesRecursive(&nr, devicesSet, 0);
```

Шестой метод – перечисление из корня сетевого окружения с использованием флагов `RESOURCEUSAGE_CONTAINER` и `RESOURCEUSAGE_CONNECTABLE`. При успешном открытии перечисления выполняется перебор всех найденных ресурсов: проверяется формат имени, извлекаются имена серверов и добавляются в множество устройств, для контейнеров выполняется рекурсивное перечисление.

```
HANDLE hEnum = nullptr;
if (WNetOpenEnumW(RESOURCE_GLOBALNET, RESOURCETYPE_ANY,
    RESOURCEUSAGE_CONTAINER | RESOURCEUSAGE_CONNECTABLE,
    nullptr, &hEnum) == NO_ERROR && hEnum) {
    NETRESOURCE* pBuffer = (NETRESOURCE*)malloc(16384);
    DWORD count = 0xFFFFFFFF;
    while (WNetEnumResourceW(hEnum, &count, pBuffer, nullptr) ==
        NO_ERROR && count > 0) {
        for (DWORD i = 0; i < count; i++) {
            if (pBuffer[i].lpRemoteName) {
                std::wstring name = pBuffer[i].lpRemoteName;
                if (name.length() >= 2 && name[0] == L'\\' && name[1] ==
                    L'\\' &&
                    name.find(L'\\', 2) == std::wstring::npos) {
                    devicesSet.insert(name);
                }
                if (pBuffer[i].dwUsage & RESOURCEUSAGE_CONTAINER) {
                    EnumerateNetworkResourcesRecursive(&pBuffer[i],
                        devicesSet, 0);
                }
            }
        }
        count = 0xFFFFFFFF;
    }
    free(pBuffer);
    WNetCloseEnum(hEnum);
}
```

Седьмой метод – добавление устройств из ARP-таблицы [6], что помогает найти устройства, которые могут отсутствовать в сетевом перечислении.

```

std::vector<std::wstring> arpDevices = GetNetworkIPsFromARP();
for (const auto& dev : arpDevices) {
    if (!dev.empty()) {
        devicesSet.insert(dev);
    }
}

```

После применения всех методов множество устройств преобразуется в вектор и возвращается как результат. Если в процессе работы алгоритма произошла ошибка (например, сетевые ресурсы недоступны или доступ запрещён), функция продолжает работу с другими методами, обеспечивая максимально полный список устройств.

### 3.2.2 Получение списка сетевых шаров

Получение списка сетевых шаров на сервере начинается с инициализации вектора для хранения шаров и создания структуры NETRESOURCEW с параметрами для поиска дисковых ресурсов.

Структура NETRESOURCEW настраивается для поиска подключаемых дисковых ресурсов (RESOURCE\_TYPE\_DISK, RESOURCEUSAGE\_CONNECTABLE) на указанном сетевом сервере.

После настройки структуры открывается перечисление сетевых ресурсов через функцию WNetOpenEnumW с указанием пути к серверу. При успешном открытии перечисления выделяется буфер для хранения найденных ресурсов.

```

std::vector<std::wstring> EnumerateNetworkShares(const std::wstring&
serverPath) {
    std::vector<std::wstring> shares;
    NETRESOURCEW nr{};
    nr.dwScope = RESOURCE_GLOBALNET;
    nr.dwType = RESOURCE_TYPE_DISK;
    nr.dwUsage = RESOURCEUSAGE_CONNECTABLE;
    nr.lpRemoteName = const_cast<wchar_t*>(serverPath.c_str());

    HANDLE hEnum = nullptr;
    if (WNetOpenEnumW(RESOURCE_GLOBALNET, RESOURCE_TYPE_DISK, 0, &nr, &
hEnum) == NO_ERROR && hEnum) {
        NETRESOURCEW* pBuffer = (NETRESOURCEW*)malloc(16384);
        DWORD count = 0xFFFFFFFF;
        while (WNetEnumResourceW(hEnum, &count, pBuffer, nullptr) ==
NO_ERROR && count > 0) {
            for (DWORD i = 0; i < count; ++i) {
                if (pBuffer[i].lpRemoteName) {
                    shares.push_back(pBuffer[i].lpRemoteName);
                }
            }
            count = 0xFFFFFFFF;
        }
        free(pBuffer);
        WNetCloseEnum(hEnum);
    }
    return shares;
}

```

Выполняется цикл перечисления ресурсов: вызывается функция `WNetEnumResourceW` для получения списка ресурсов, при успешном выполнении выполняется перебор всех найденных ресурсов, и имена удалённых ресурсов (`lpRemoteName`) добавляются в вектор шаров.

Цикл продолжается до тех пор, пока не будет получен код ошибки `ERROR_NO_MORE_ITEMS`, что означает, что все ресурсы перечислены.

После завершения перечисления буфер освобождается, перечисление закрывается через `WNetCloseEnum`, и вектор шаров возвращается как результат.

Если в процессе работы алгоритма произошла ошибка (например, сервер недоступен, доступ запрещён или перечисление не удалось открыть), функция возвращает пустой вектор шаров.

### 3.2.3 Поиск файлов и шаров на сетевых устройствах

Поиск файлов и шаров на сетевых устройствах начинается с удаления placeholder-элементов из узла сетевого устройства в дереве каталогов.

После удаления placeholder-элементов функция проверяет данные узла на существование и проверяет, был ли узел уже расширен ранее, чтобы избежать повторной загрузки сетевых ресурсов.

Если узел ещё не расширен, проверяется формат сетевого пути. В этом случае вызывается функция `EnumerateNetworkShares` для получения списка всех доступных шаров на этом сервере.

```
void ExpandNetworkDevice(HWND hTree, HTREEITEM hItem) {
    TVITEMW tvi{}; tvi.mask = TVIF_PARAM; tvi.hItem = hItem;
    TreeView_GetItem(hTree, &tvi);
    Filesystem::NodeData *data = reinterpret_cast<Filesystem::NodeData*>
        (tvi.lParam);
    if (!data || data->expanded) return;

    if (data->path.length() >= 2 && data->path[0] == L'\\' &&
        data->path[1] == L'\\' && data->path.find(L'\\', 2) == std::
            wstring::npos) {
        std::vector<std::wstring> shares = EnumerateNetworkShares(data->
            path);
        for (const auto& share : shares) {
            size_t lastSlash = share.find_last_of(L'\\');
            std::wstring displayName = (lastSlash != std::wstring::npos)
                ?
                share.substr(lastSlash + 1) : share;
            bool hasChildren = Filesystem::HasSubdirs(share);
            Filesystem::InsertTreeItem(hTree, hItem, displayName, share,
                hasChildren);
        }
        data->expanded = true;
    } else {
        Filesystem::ExpandTreeItem(hTree, hItem);
    }
}
```



Для каждого найденного шара извлекается отображаемое имя (последний сегмент пути после последней обратной косой черты), проверяется наличие подкаталогов в шаре, и шар добавляется в дерево с соответствующей пометкой о наличии дочерних элементов.

Если путь содержит третью обратную косую черту, это означает, что путь указывает на конкретную шару или подкаталог, и для его обработки вызывается стандартный алгоритм расширения локального узла дерева через функцию `ExpandTreeItem`.

После завершения обработки узел помечается как расширенный, чтобы предотвратить повторную загрузку при следующем раскрытии.

Если в процессе работы алгоритма произошла ошибка (например, сетевой сервер недоступен, доступ запрещён или шары не найдены), функция завершается без добавления элементов, и узел остаётся в исходном состоянии.

### **3.3 Изменение настроек**

К функционалу изменения настроек программного средства можно отнести следующие компоненты:

- получение настроек;
- изменение настроек;

#### **3.3.1 Получение настроек**

Получение настроек начинается с открытия ключа реестра `Windows`, в котором хранятся сохранённые параметры конфигурации приложения.

После успешного открытия ключа реестра функция последовательно читает три параметра: тему оформления, размер шрифта и имя шрифта. Для каждого параметра выполняется проверка успешности чтения и валидация значения.

При чтении темы оформления проверяется, что значение находится в допустимом диапазоне (от `System` до `Dark`), и если значение валидно, оно устанавливается в глобальную переменную.

При чтении размера шрифта проверяется, что значение находится в диапазоне от 8 до 72 пунктов, и если значение валидно, оно устанавливается в глобальную переменную.

При чтении имени шрифта проверяется успешность чтения строкового значения, и если чтение успешно, имя шрифта устанавливается в глобальную переменную.

После обработки всех параметров ключ реестра закрывается, и вызывается функция `UpdateThemeColors` для применения загруженных настроек темы к цветовой схеме интерфейса.

Если в процессе работы алгоритма произошла ошибка (например, ключ реестра не существует, доступ запрещён или значение невалидно),

функция использует значения по умолчанию (тема System, размер шрифта 12, имя шрифта "Segoe UI") и продолжает работу.

```
void Load() {
    HKEY hKey;
    if (RegOpenKeyExW(HKEY_CURRENT_USER, L"Software\\KursWork\\
        FileManager",
                    0, KEY_QUERY_VALUE, &hKey) == ERROR_SUCCESS) {
        DWORD theme = (DWORD)ThemeChoice::System;
        DWORD sz = sizeof(theme);
        if (RegGetValueW(hKey, nullptr, L"Theme", RRF_RT_REG_DWORD,
            nullptr, &theme, &sz) == ERROR_SUCCESS) {
            if (theme <= (DWORD)ThemeChoice::Dark) {
                g_themeChoice = (ThemeChoice)theme;
            }
        }
        DWORD size = 12;
        sz = sizeof(size);
        if (RegGetValueW(hKey, nullptr, L"FontSize", RRF_RT_REG_DWORD,
            nullptr, &size, &sz) == ERROR_SUCCESS) {
            if (size >= 8 && size <= 72) {
                g_fontPt = (int)size;
            }
        }
        wchar_t buf[128];
        DWORD bytes = sizeof(buf);
        if (RegGetValueW(hKey, nullptr, L"FontFace", RRF_RT_REG_SZ,
            nullptr, buf, &bytes) == ERROR_SUCCESS) {
            g_fontFace = buf;
        }
        RegCloseKey(hKey);
    }
    UpdateThemeColors();
}
```

### 3.3.2 Изменение настроек

Изменение настроек начинается с обработки команды пользователя из меню приложения. В зависимости от выбранной команды определяется тип изменяемой настройки (тема, имя шрифта или размер шрифта).

При изменении темы оформления проверяется идентификатор выбранной команды меню, и в зависимости от него устанавливается соответствующее значение темы (System, Light или Dark). После установки темы вызывается функция Save для сохранения изменений в реестр, и пользователю отображается сообщение о необходимости перезагрузки приложения.

При изменении имени шрифта проверяется идентификатор выбранной команды меню, и в зависимости от него устанавливается соответствующее имя шрифта ("Segoe UI", "Consolas" или "Courier New"). После установки имени шрифта вызывается функция ApplyFontToControls для применения нового шрифта ко всем элементам интерфейса, обновляются отметки в меню, и вызывается функция Save для сохранения изменений.

При изменении размера шрифта проверяется идентификатор выбранной команды меню или клавиатурной комбинации. Если это команда увеличения размера, проверяется, что новый размер не превышает 72 пункта, и размер увеличивается на 1. Если это команда уменьшения размера, проверяется, что новый размер не меньше 8 пунктов, и размер уменьшается на 1. После установки размера шрифта вызывается функция `ApplyFontToControls` для применения нового размера ко всем элементам интерфейса, обновляются отметки в меню, и вызывается функция `Save` для сохранения изменений.

```

case ID_MENU_THEME_SYSTEM:
case ID_MENU_THEME_LIGHT:
case ID_MENU_THEME_DARK: {
    if (LOWORD(wParam) == ID_MENU_THEME_SYSTEM) {
        Settings::SetTheme(ThemeChoice::System);
    } else if (LOWORD(wParam) == ID_MENU_THEME_LIGHT) {
        Settings::SetTheme(ThemeChoice::Light);
    } else {
        Settings::SetTheme(ThemeChoice::Dark);
    }
    Settings::Save();
    return 0;
}
case ID_MENU_FONT_SEGOE:
case ID_MENU_FONT_CONSOLAS:
case ID_MENU_FONT_COURIER: {
    if (LOWORD(wParam) == ID_MENU_FONT_SEGOE) {
        Settings::SetFontFace(L"Segoe UI");
    } else if (LOWORD(wParam) == ID_MENU_FONT_CONSOLAS) {
        Settings::SetFontFace(L"Consolas");
    } else {
        Settings::SetFontFace(L"Courier New");
    }
    UI::ApplyFontToControls(hwnd, *controls);
    UI::UpdateMenuChecks(hwnd);
    Settings::Save();
    return 0;
}
case ID_CMD_FONTSIZE_INC: {
    Settings::SetFontSize(std::min(72, Settings::GetFontSize() + 1));
    UI::ApplyFontToControls(hwnd, *controls);
    UI::UpdateMenuChecks(hwnd);
    Settings::Save();
    return 0;
}
case ID_CMD_FONTSIZE_DEC: {
    Settings::SetFontSize(std::max(8, Settings::GetFontSize() - 1));
    UI::ApplyFontToControls(hwnd, *controls);
    UI::UpdateMenuChecks(hwnd);
    Settings::Save();
    return 0;
}

```

Если в процессе работы алгоритма произошла ошибка (например, не удалось сохранить настройки в реестр), изменения применяются только к текущей сессии приложения и не сохраняются для следующего запуска.

## 4 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

В процессе разработки программного средства производилось регулярное тестирование на промежуточных этапах. Анализировалось и устранялось большое количество проблем часть из которых описана в данном разделе далее.

### 4.1 Ошибка компиляции из-за отсутствующего заголовочного файла

При добавлении функции `GetNetworkScanMutex()` в заголовочный файл `ui.h` возникла ошибка компиляции в нескольких модулях. Функция возвращала ссылку на `std::mutex`, но заголовочный файл `<mutex>` не был включён в `ui.h`, что приводило к ошибке компиляции во всех модулях, которые включали `ui.h`.

Информацию об ошибке можно рассмотреть ниже.

```
//psf/Home/Documents/university/5_term/CurseWork/file-manager/src/ui/
layout.cpp: In function 'void UI::LayoutChildren(HWND, const
    UIControls&, const UIState&)':
//psf/Home/Documents/university/5_term/CurseWork/file-manager/src/ui/
layout.cpp:23:15: error: 'mutex' in namespace 'std' does not name a
    type
23 |         std::mutex& mtx = UI::GetNetworkScanMutex(hwnd);
    |         ^~~~~~
In file included from //psf/Home/Documents/university/5_term/CurseWork/
    file-manager/src/ui/layout.cpp:10:
//psf/Home/Documents/university/5_term/CurseWork/file-manager/include/ui
    .h:50:8: note: 'std::mutex' has not been declared
50 |         std::mutex& GetNetworkScanMutex(HWND hwnd);
    |         ^~~~~~
```

Проблема была решена добавлением директивы `#include <mutex>` в начало файла `ui.h`. Это обеспечило объявление типа `std::mutex` во всех модулях, которые включают заголовочный файл `ui.h`.

```
#include <mutex>

namespace UI {
    std::mutex& GetNetworkScanMutex(HWND hwnd);
}
```

После добавления `#include <mutex>` все ошибки компиляции были устранены. Все модули успешно компилируются, и функция `GetNetworkScanMutex()` корректно работает во всех местах использования. Это подтвердило важность включения всех необходимых заголовочных файлов в заголовочные файлы, которые используются другими модулями.

## 4.2 Ошибка сборки из-за неправильных флагов компилятора

При попытке собрать Release версию проекта возникла ошибка компиляции. В файле `CMakeLists.txt` были указаны флаги оптимизации для компилятора MSVC (`/O2`, `/DNDEBUG`), но проект собирался с помощью компилятора GCC/MinGW, который использует другой синтаксис флагов.

Рассмотреть ошибку можно ниже.

```
c++.exe: warning: /O2: linker input file unused because linking not done
c++.exe: error: /O2: linker input file not found: No such file or
directory
c++.exe: warning: /DNDEBUG: linker input file unused because linking not
done
c++.exe: error: /DNDEBUG: linker input file not found: No such file or
directory
```

Для решения проблемы было необходимо добавить проверку типа компилятора в `CMakeLists.txt` и применять соответствующие флаги в зависимости от используемого компилятора. Для MSVC используются флаги `/O2` и `/DNDEBUG`, а для GCC/MinGW - флаги `-O2`, `-DNDEBUG` и `-s` (для удаления символов).

```
# Release build optimizations
if(CMAKE_BUILD_TYPE STREQUAL "Release")
    if(MSVC)
        # MSVC compiler flags
        set(CMAKE_CXX_FLAGS_RELEASE "${CMAKE_CXX_FLAGS_RELEASE} /O2 /
DNDEBUG")
        set(CMAKE_EXE_LINKER_FLAGS_RELEASE "${
CMAKE_EXE_LINKER_FLAGS_RELEASE} /OPT:REF /OPT:ICF")
    else()
        # GCC/MinGW compiler flags
        set(CMAKE_CXX_FLAGS_RELEASE "${CMAKE_CXX_FLAGS_RELEASE} -O2 -
DNDEBUG")
        set(CMAKE_EXE_LINKER_FLAGS_RELEASE "${
CMAKE_EXE_LINKER_FLAGS_RELEASE} -s")
    endif()
endif()
```

После внесения изменений в `CMakeLists.txt` Release версия проекта успешно собирается как с компилятором MSVC, так и с GCC/MinGW. Флаги оптимизации применяются корректно в зависимости от используемого компилятора, что обеспечивает оптимальный размер и производительность исполняемого файла. Проект теперь может быть собран в Release режиме на различных системах сборки без ошибок компиляции.

## 5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

### 5.1 Интерфейс программного средства

В интерфейсе программного средства имеются схожести со стандартным проводником Windows, так как использовалась общая API к доступу операционной системы.

#### 5.1.1 Главная окно

Главное окно – основное рабочее пространство приложения, с которым пользователь взаимодействует при работе с файловой системой.

Главное окно состоит из следующих компонентов:

- панель инструментов;
- дерево каталогов;
- список файлов;
- строка состояния;
- меню приложения.

Рассмотреть главное окно программного средства можно на рисунке 5.1.

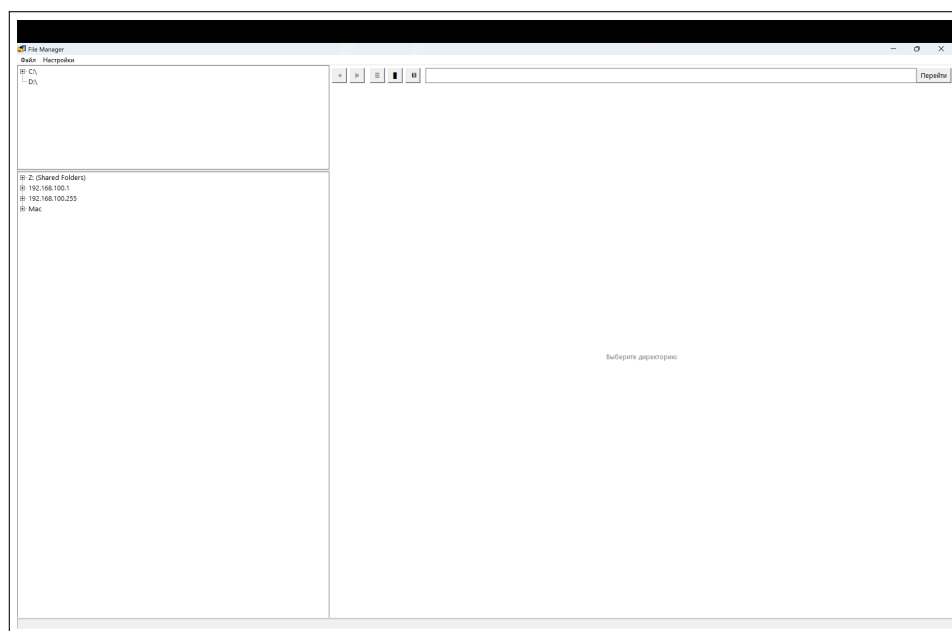


Рисунок 5.1 – Главное окно программного средства

Панель инструментов расположена в верхней части окна и содержит кнопки навигации ("Назад", "Вперёд"), поле ввода пути для быстрого перехода в нужную директорию, кнопку "Перейти" и кнопки переключения режимов отображения файлов ("Детали", "Крупные значки", "Мелкие значки"). Панель инструментов позволяет пользователю быстро выполнять основные операции навигации и изменения представления файлов.

Дерево каталогов расположено в левой части окна и отображает иерархическую структуру файловой системы. В дереве отображаются все

доступные локальные диски, сетевые устройства и их подкаталоги. Пользователь может раскрывать и сворачивать узлы дерева для навигации по файловой системе. Дерево каталогов имеет две вкладки: "Локальные диски" для работы с локальной файловой системой и "Сеть" для работы с сетевыми ресурсами.

Список файлов расположен в правой части окна и отображает содержимое выбранной директории. Файлы и папки отображаются в виде таблицы с колонками "Имя", "Тип" и "Изменён", либо в виде иконок в зависимости от выбранного режима отображения. Каждый элемент списка имеет соответствующую системную иконку, которая помогает визуально различать типы файлов.

Строка состояния расположена в нижней части окна и отображает информацию о текущей директории: количество файлов и папок в формате "Файлов: X, Папок: Y". Строка состояния автоматически обновляется при переходе в другую директорию.

Меню приложения расположено в верхней части окна под заголовком и содержит пункты "Файл" и "Настройки". Пункт "Файл" содержит команды "О программе" и "Выход", а пункт "Настройки" позволяет изменять тему оформления, шрифт и размер шрифта приложения.

### 5.1.2 О программе

Окно "О программе" – диалоговое окно, которое отображает информацию о программном средстве, его версии и разработчике.

Окно "О программе" состоит из следующих частей:

- заголовок окна;
- информация о программе;
- кнопка закрытия.

Внешний вид окна о программе можно увидеть на рисунке 5.2.

Страница пользователя состоит из следующих частей:

- шапка страницы;
- меню;

Внешний вид окна о программе можно увидеть на рисунке 5.2.

Заголовок окна содержит название программного средства "File Manager" и стандартные кнопки управления окном Windows (свернуть, развернуть, закрыть). Окно "О программе" является модальным диалоговым окном, что означает, что пользователь должен закрыть его перед продолжением работы с основным приложением.

Информация о программе отображается в центральной части окна и содержит название приложения, описание функциональности, информацию о версии и разработчике. Окно "О программе" позволяет пользователю получить базовую информацию о программном средстве и его назначении.

Кнопка закрытия позволяет пользователю закрыть окно "О программе" и вернуться к работе с основным приложением. Окно также

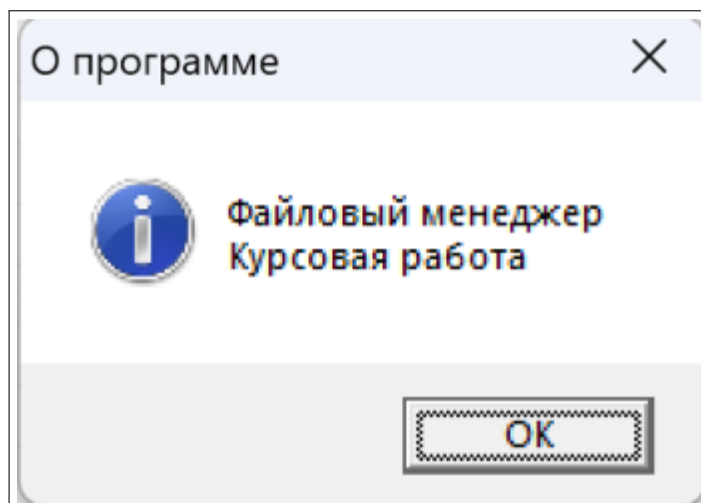


Рисунок 5.2 – Окно «О программе»

можно закрыть стандартными способами Windows: нажатием клавиши Escape или щелчком по кнопке закрытия в заголовке окна.

## 5.2 Управление программным средством

### 5.2.1 Навигация по файловой системе

Перед началом работы с программным средством пользователю необходимо ознакомиться с интерфейсом приложения. Основное окно приложения разделено на две части: левая панель содержит дерево каталогов, а правая панель отображает содержимое выбранной директории.

Для навигации по файловой системе пользователь может использовать дерево каталогов в левой части окна. При первом запуске приложения в дереве отображаются все доступные локальные диски (C: D: и т.д.). Для просмотра содержимого диска необходимо щёлкнуть на его название в дереве. При этом в правой панели отобразится список файлов и папок, находящихся в корне выбранного диска.

Для просмотра содержимого подкаталога необходимо раскрыть соответствующий узел в дереве каталогов, щёлкнув на значок ”+” или двойным кликом по названию папки. При раскрытии узла приложение автоматически загружает список подкаталогов и отображает их в дереве. Если у папки есть подкаталоги, рядом с её названием отображается значок ”+”, который можно использовать для раскрытия.

### 5.2.2 Просмотр содержимого директории

Для просмотра содержимого директории пользователь может выбрать директорию в дереве каталогов или ввести путь к директории в поле ввода пути, расположенном в верхней части окна. После ввода пути необходимо нажать кнопку ”Перейти” или клавишу Enter для перехода в указанную директорию.



Содержимое директории отображается в правой панели в виде таблицы с тремя колонками:

- Имя – название файла или папки;
- Тип – тип элемента (Dir для директорий, File для файлов);
- Изменён – дата и время последнего изменения файла или директории.

Каждый элемент в списке отображается с соответствующей системной иконкой, которая помогает визуально различать типы файлов. Директории отображаются с иконкой папки, а файлы – с иконкой, соответствующей их типу (изображения, документы, исполняемые файлы и т.д.).

Для сортировки элементов по любой колонке пользователь может щёлкнуть на заголовок колонки. Повторный щелчок изменяет направление сортировки (по возрастанию или по убыванию).

### **5.2.3 Открытие файлов и переход в директории**

Для открытия файла или перехода в директорию пользователь может выполнить двойной клик по соответствующему элементу в списке файлов. Если выбранный элемент является директорией, приложение автоматически переходит в эту директорию и отображает её содержимое в правой панели, а также обновляет дерево каталогов, выделяя текущую директорию.

Если выбранный элемент является файлом, приложение пытается открыть файл с помощью ассоциированного с ним приложения операционной системы. Если для файла не найдено ассоциированное приложение или произошла ошибка при открытии, пользователю отображается сообщение об ошибке "Не удалось открыть файл".

Для навигации по истории посещённых директорий пользователь может использовать кнопки "Назад" и "Вперёд", расположенные в верхней части окна. Кнопка "Назад" позволяет вернуться к ранее посещённой директории, а кнопка "Вперёд" – перейти к следующей директории в истории после использования кнопки "Назад".

### **5.2.4 Работа с сетевыми устройствами**

Особенностью программного средства является возможность работы с сетевыми устройствами и общими ресурсами в локальной сети. Для просмотра сетевых устройств пользователь может переключиться на вкладку "Сеть" в нижней части левой панели.

При переключении на вкладку "Сеть" приложение автоматически начинает поиск доступных сетевых устройств в локальной сети. Поиск выполняется с использованием нескольких методов:

- перечисление сетевых дисков, сопоставленных буквам (Z:, Y: и т.д.);
- рекурсивное перечисление всех сетевых ресурсов;
- перечисление рабочих групп и доменов;
- перечисление серверов и дисковых шаров;

- поиск устройств через ARP-таблицу.

Найденные сетевые устройства отображаются в дереве каталогов на вкладке "Сеть". Для просмотра содержимого сетевого устройства пользователь может щёлкнуть на его название. Если устройство является сетевым сервером, приложение автоматически получает список всех доступных шаров (общих папок) на этом сервере и отображает их в дереве.

При первом обращении к сетевому ресурсу приложение может отобразить сообщение "Загрузка..." или "Подключение...", так как сетевым ресурсам требуется время для инициализации и установления соединения. После успешного подключения содержимое сетевого ресурса отображается аналогично локальным директориям.

### 5.2.5 Настройки приложения

Особенностью программного средства является гибкая настройка внешнего вида. Пользователю доступны следующие настройки через меню "Настройки":

- Тема оформления – выбор цветовой схемы интерфейса (Системная, Светлая или Тёмная);

- Имя шрифта – выбор шрифта для отображения текста (Segoe UI, Consolas или Courier New);

- Размер шрифта – изменение размера шрифта;

Для изменения темы оформления пользователь может выбрать пункт меню "Настройки" → "Тема" и выбрать одну из доступных тем. После выбора темы приложение сохраняет настройки и отображает сообщение о том, что изменения вступят в силу после перезагрузки приложения. Для изменения шрифта пользователь может выбрать пункт меню "Настройки" → "Шрифт" и выбрать одно из доступных имён шрифтов. Изменение шрифта применяется немедленно ко всем элементам интерфейса приложения. Для изменения размера шрифта пользователь может выбрать пункт меню "Настройки" → "Размер шрифта" и выбрать один из предустановленных размеров (10, 12, 14 или 16 пунктов), либо использовать клавиатурные комбинации Ctrl+Plus для увеличения размера или Ctrl+Minus для уменьшения размера.

Внешний вид окна настроек можно увидеть на рисунке 5.3.

Все настройки автоматически сохраняются в реестре Windows и восстанавливаются при следующем запуске приложения. Это позволяет пользователю настроить внешний вид приложения один раз и не изменять настройки при каждом запуске.

### 5.2.6 Режимы отображения файлов

Программное средство поддерживает несколько режимов отображения файлов в правой панели. Пользователь может переключаться между режимами с помощью кнопок в верхней части окна:

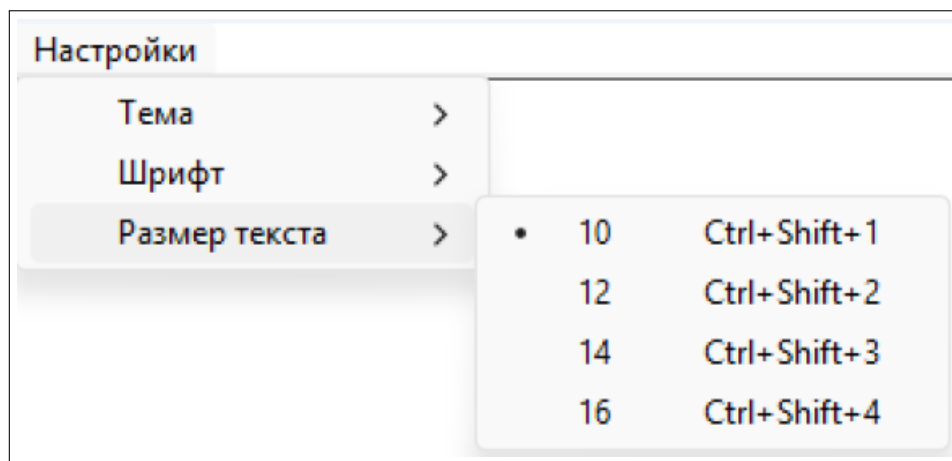


Рисунок 5.3 – Внешний вид настроек приложения

- Детали – отображение файлов в виде таблицы с колонками (Имя, Тип, Изменён);
- Крупные значки – отображение файлов в виде крупных иконок с подписями;
- Мелкие значки – отображение файлов в виде мелких иконок с подписями.

Текущий режим отображения сохраняется и восстанавливается при следующем запуске приложения.

#### 5.2.7 Строка состояния

В нижней части окна приложения расположена строка состояния, которая отображает информацию о текущей директории. Строка состояния показывает количество файлов и папок в текущей директории в формате "Файлов: X, Папок: Y". Если директория пуста, в строке состояния отображается текст "Готово".

Строка состояния автоматически обновляется при переходе в другую директорию или при изменении содержимого текущей директории.

## ЗАКЛЮЧЕНИЕ

В данном курсовом проекте было разработано программное средство для управления файловой системой, которое позволяет эффективно навигироваться по локальным и сетевым ресурсам, просматривать содержимое директорий и управлять файлами. В ходе работы над проектом были изучены основные аспекты разработки программного обеспечения для платформы Windows, включая анализ требований, проектирование модульной архитектуры, реализацию с использованием Win32 API и тестирование.

Приложение создано с использованием современных технологий и инструментов разработки на языке C++, что позволяет обеспечить его высокую производительность и надежность. При разработке были реализованы следующие основные функции программного средства:

- модуль для работы с файловой системой;
- модуль для работы с сетевыми устройствами;
- модуль управления настройками приложения;
- модуль пользовательского интерфейса;
- модуль утилитарных функций;
- алгоритм загрузки настроек из реестра Windows;
- алгоритм расширения узлов дерева каталогов;
- алгоритм обновления строки состояния директории;
- алгоритм обработки файлов директории;
- алгоритм добавления корневых дисков в дерево;
- алгоритм поиска сетевых устройств;
- алгоритм получения списка сетевых шаров;

Для достижения поставленных целей были использованы современные подходы и методологии разработки программного обеспечения, такие как модульная архитектура, принцип единственной ответственности (Single Responsibility Principle) и инкапсуляция данных. Это позволило обеспечить высокое качество программного продукта, его поддерживаемость и расширяемость.

В ходе работы над проектом были выявлены некоторые проблемы и ограничения, которые могут быть устранены в будущем. Например, добавление функциональности копирования и перемещения файлов, реализация поиска файлов по имени и содержимому, улучшение обработки больших директорий с тысячами файлов, оптимизация работы с сетевыми ресурсами при медленном соединении.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] CLion documentation [Электронный ресурс]. — Режим доступа : <https://www.jetbrains.com/clion/learn/>. — Дата доступа : 15.09.2025.
- [2] C++ documentation [Электронный ресурс]. — Режим доступа : <https://learn.microsoft.com/ru-ru/cpp>. — Дата доступа : 25.09.2025.
- [3] MinGW documentation [Электронный ресурс]. — Режим доступа : <https://www.mingw-w64.org/>. — Дата доступа : 27.09.2025.
- [4] Martin, Robert C. Clean Architecture: A Craftsman's Guide to Software Structure and Design / Robert C. Martin. — Prentice Hall, 2017.
- [5] Win32 API documentation [Электронный ресурс]. — Режим доступа : <https://learn.microsoft.com/ru-ru/windows/win32/api/>. — Дата доступа : 15.10.2025.
- [6] IP Helper API documentation [Электронный ресурс]. — Режим доступа : <https://learn.microsoft.com/ru-ru/windows/win32/api/iphlpapi/>. — Дата доступа : 15.10.2025.
- [7] БГУИР, Минск. — Стандарт предприятия. Дипломные проекты (работы). Общие требования : СТП 01-2017, 2017. — Введ. 01.01.2018.

## ПРИЛОЖЕНИЕ А

### (обязательное)

### Исходный код

#### File System

```
#ifndef UNICODE
#define UNICODE
#endif
#ifndef _UNICODE
#define _UNICODE
#endif

#include "../include/filesystem.h"
#include "../include/utils.h"
#include <windows.h>
#include <commctrl.h>
#include <shellapi.h>
#include <algorithm>

namespace Filesystem {
    bool HasSubdirs(const std::wstring &path) {
        std::wstring pattern = Utils::JoinPath(path, L"*");
        WIN32_FIND_DATA fd{};
        HANDLE h = FindFirstFileW(pattern.c_str(), &fd);
        if (h == INVALID_HANDLE_VALUE) return false;
        bool result = false;
        do {
            if ((fd.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY) == 0)
                continue;
            if (wcscmp(fd.cFileName, L"..") == 0 || wcscmp(fd.cFileName,
                L"..") == 0) continue;
            result = true;
            break;
        } while (FindNextFileW(h, &fd));
        FindClose(h);
        return result;
    }

    void PopulateListForDirectory(HWND hList, const std::wstring &
        dirPath,
                                std::vector<ListItem> &items, HWND
                                hEmptyLabel) {
        ListView_DeleteAllItems(hList);
        items.clear();

        // Normalize path - resolve any symbolic links or short paths
        std::wstring normalizedPath = dirPath;
        wchar_t longPath[MAX_PATH + 1] = {0};
        DWORD longPathLen = GetLongPathNameW(dirPath.c_str(), longPath,
            MAX_PATH);
        if (longPathLen > 0 && longPathLen <= MAX_PATH) {
            normalizedPath = longPath;
        }

        // Ensure path ends with backslash for directories (except root
        // like C:\)
        if (normalizedPath.length() > 3 && normalizedPath.back() != L
            '\\') {
            normalizedPath += L"\\";
        }
    }
}
```

```

}

// Check if path exists - try original path first, then
normalized
DWORD attribs = GetFileAttributesW(normalizedPath.c_str());
if (attribs == INVALID_FILE_ATTRIBUTES && normalizedPath !=
dirPath) {
    // Try original path if normalized failed
    attribs = GetFileAttributesW(dirPath.c_str());
    if (attribs != INVALID_FILE_ATTRIBUTES) {
        normalizedPath = dirPath;
    }
}

if (attribs == INVALID_FILE_ATTRIBUTES) {
    // Path doesn't exist - check if it's a network path that
    might need time to connect
    if (hEmptyLabel) {
        // For network paths, show Подключение"... instead of
        Файл" не найден "
        bool isNetworkPath = (normalizedPath.length() >= 2 &&
normalizedPath[0] == L'\\' && normalizedPath[1] == L
'\\');
        if (isNetworkPath) {
            SetWindowTextW(hEmptyLabel, LПодключение"...");
        } else {
            SetWindowTextW(hEmptyLabel, LФайл" не найден ");
        }
        ShowWindow(hEmptyLabel, SW_SHOW);
        SetWindowPos(hEmptyLabel, HWND_TOP, 0, 0, 0, 0,
SWP_NOMOVE | SWP_NOSIZE);
        InvalidateRect(hEmptyLabel, nullptr, TRUE);
    }
    // Hide list view to show only the message
    ShowWindow(hList, SW_HIDE);
    return;
}

// Check if it's a directory
if (!(attribs & FILE_ATTRIBUTE_DIRECTORY)) {
    // It's a file, not a directory - show Файл" пуст" message
    if (hEmptyLabel) {
        SetWindowTextW(hEmptyLabel, LФайл" пуст");
        ShowWindow(hEmptyLabel, SW_SHOW);
        SetWindowPos(hEmptyLabel, HWND_TOP, 0, 0, 0, 0,
SWP_NOMOVE | SWP_NOSIZE);
        InvalidateRect(hEmptyLabel, nullptr, TRUE);
    }
    // Hide list view to show only the message
    ShowWindow(hList, SW_HIDE);
    InvalidateRect(hList, nullptr, TRUE);
    UpdateWindow(hList);
    RedrawWindow(hList, nullptr, nullptr, RDW_INVALIDATE |
RDW_UPDATENOW | RDW_ERASE);
    return;
}

// Check if it's a network path - these may need time to load
bool isNetworkPath = (normalizedPath.length() >= 2 &&
normalizedPath[0] == L'\\' && normalizedPath[1] == L'\\');

```

```

std::wstring pattern = Utils::JoinPath(normalizedPath, L"*");
WIN32_FIND_DATA fd{};
HANDLE h = FindFirstFileW(pattern.c_str(), &fd);

// If first attempt fails, show loading and retry after a delay
// This handles cases where directory is still loading
// (especially network paths and special folders)
if (h == INVALID_HANDLE_VALUE) {
    DWORD error = GetLastError();

    // Show loading message while we retry
    if (hEmptyLabel) {
        SetWindowTextW(hEmptyLabel, L"Загрузка...");
        ShowWindow(hEmptyLabel, SW_SHOW);
        SetWindowPos(hEmptyLabel, HWND_TOP, 0, 0, 0, 0,
            SWP_NOMOVE | SWP_NOSIZE);
        ShowWindow(hList, SW_HIDE);
        InvalidateRect(hEmptyLabel, nullptr, TRUE);
        UpdateWindow(hList);
    }

    // Retry after delay - network paths need more time, local
    // paths also need time for special folders
    // Special folders like Downloads might need time to
    // initialize
    int delay = isNetworkPath ? 1500 : 800; // 1.5 seconds for
        network, 800ms for local (handles special folders)

    // Use a simple delay that doesn't block too much
    // But we need to process messages to keep UI responsive
    DWORD startTime = GetTickCount();
    while ((GetTickCount() - startTime) < delay) {
        // Process a few messages to keep UI responsive
        MSG msg;
        while (PeekMessageW(&msg, nullptr, 0, 0, PM_REMOVE)) {
            TranslateMessage(&msg);
            DispatchMessageW(&msg);
            if ((GetTickCount() - startTime) >= delay) break;
        }
        if ((GetTickCount() - startTime) < delay) {
            Sleep(50); // Small sleep to avoid busy waiting
        }
    }

    // Try again
    h = FindFirstFileW(pattern.c_str(), &fd);
}

if (h == INVALID_HANDLE_VALUE) {
    // Directory is inaccessible or empty after retry
    DWORD error = GetLastError();
    if (hEmptyLabel) {
        if (error == ERROR_ACCESS_DENIED || error ==
            ERROR_SHARING_VIOLATION) {
            SetWindowTextW(hEmptyLabel, L"Доступ запрещён");
        } else if (isNetworkPath) {
            // For network paths, even after retry, show loading
            // - they might still be connecting
            SetWindowTextW(hEmptyLabel, L"Загрузка...");
        }
    }
}

```



```

    } else if (error == ERROR_FILE_NOT_FOUND || error ==
        ERROR_NO_MORE_FILES) {
        // For local paths after retry, if still empty, show
        // empty message
        SetWindowTextW(hEmptyLabel, L"Здесь пусто");
    } else {
        // Other local errors - assume empty
        SetWindowTextW(hEmptyLabel, L"Здесь пусто");
    }
    ShowWindow(hEmptyLabel, SW_SHOW);
    SetWindowPos(hEmptyLabel, HWND_TOP, 0, 0, 0, 0,
        SWP_NOMOVE | SWP_NOSIZE);
    InvalidateRect(hEmptyLabel, nullptr, TRUE);
}
// Hide list view to show only the message
ShowWindow(hList, SW_HIDE);
InvalidateRect(hList, nullptr, TRUE);
UpdateWindow(hList);
RedrawWindow(hList, nullptr, nullptr, RDW_INVALIDATE |
    RDW_UPDATENOW | RDW_ERASE);
return;
}

int idx = 0;
do {
    std::wstring name = fd.cFileName;
    if (name == L"." || name == L"..") continue;
    bool isDir = (fd.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY
        ) != 0;

    ListItem ri{ name, Utils::JoinPath(normalizedPath, name),
        isDir };
    items.push_back(ri);

    LVITEMW item{};
    SHFILEINFOW sfi{};
    UINT attrs = isDir ? FILE_ATTRIBUTE_DIRECTORY :
        FILE_ATTRIBUTE_NORMAL;
    SHGetFileInfoW(ri.fullPath.c_str(), attrs, &sfi, sizeof(sfi)
        ,
        SHGFI_SYSICONINDEX | SHGFI_USEFILEATTRIBUTES |
        SHGFI_SMALLICON);

    item.mask = LVIF_TEXT | LVIF_PARAM | LVIF_IMAGE;
    item.iItem = idx;
    item.pszText = const_cast<wchar_t*>(name.c_str());
    item.lParam = static_cast<LPARAM>(idx);
    item.iImage = sfi.iIcon;
    ListView_InsertItem(hList, &item);

    std::wstring typeText = isDir ? L"Dir" : L"File";
    ListView_SetItemText(hList, idx, 1, const_cast<wchar_t*>
        (typeText.c_str()));

    std::wstring mod = Utils::FileTimeToWString(fd.
        ftLastWriteTime);
    ListView_SetItemText(hList, idx, 2, const_cast<wchar_t*>(mod.
        c_str()));
    ++idx;
} while (FindNextFileW(h, &fd));

```

```

FindClose(h);

if (hEmptyLabel) {
    if (items.empty()) {
        // Directory is empty - show message and hide list view
        SetWindowTextW(hEmptyLabel, LЗдесь" пусто");
        ShowWindow(hEmptyLabel, SW_SHOW);
        SetWindowPos(hEmptyLabel, HWND_TOP, 0, 0, 0, 0,
            SWP_NOMOVE | SWP_NOSIZE);
        InvalidateRect(hEmptyLabel, nullptr, TRUE);
        // Hide list view to show only the message
        ShowWindow(hList, SW_HIDE);
    } else {
        // Directory has items - show list view and hide message
        ShowWindow(hEmptyLabel, SW_HIDE);
        ShowWindow(hList, SW_SHOW);
    }
} else {
    // If no empty label, always show list view
    ShowWindow(hList, SW_SHOW);
}

InvalidateRect(hList, nullptr, TRUE);
UpdateWindow(hList);
RedrawWindow(hList, nullptr, nullptr, RDW_INVALIDATE |
    RDW_UPDATENOW | RDW_ERASE);
}

HTREEITEM InsertTreeItem(HWND hTree, HTREEITEM hParent,
    const std::wstring &text, const std::
        wstring &fullPath,
    bool placeholderChild) {
    TVINSERTSTRUCTW ins{};
    ins.hParent = hParent;
    ins.hInsertAfter = TVI_LAST;
    ins.item.mask = TVIF_TEXT | TVIF_PARAM;
    ins.item.pszText = const_cast<wchar_t*>(text.c_str());
    NodeData *data = new NodeData{fullPath, false};
    ins.item.lParam = reinterpret_cast<LPARAM>(data);
    HTREEITEM hItem = TreeView_InsertItem(hTree, &ins);
    if (placeholderChild) {
        TVINSERTSTRUCTW child{};
        child.hParent = hItem;
        child.hInsertAfter = TVI_LAST;
        child.item.mask = TVIF_TEXT;
        child.item.pszText = const_cast<wchar_t*>(L"...");
        TreeView_InsertItem(hTree, &child);
    }
    return hItem;
}

void ExpandTreeItem(HWND hTree, HTREEITEM hItem) {
    // Remove placeholder children
    HTREEITEM hChild = TreeView_GetChild(hTree, hItem);
    while (hChild) {
        TVITEMW tvi{}; tvi.mask = TVIF_TEXT; tvi.hItem = hChild;
        wchar_t buf[8]; tvi.pszText = buf; tvi.cchTextMax = 8;
        TreeView_GetItem(hTree, &tvi);
        if (wcscmp(buf, L"...") == 0) {
            TreeView_DeleteItem(hTree, hChild);
        }
        hChild = TreeView_GetChild(hTree, hChild);
    }
}

```

```

        break;
    }
    hChild = TreeView_GetNextSibling(hTree, hChild);
}

TVITEMW tvi{}; tvi.mask = TVIF_PARAM; tvi.hItem = hItem;
TreeView_GetItem(hTree, &tvi);
NodeData *data = reinterpret_cast<NodeData*>(tvi.lParam);
if (!data || data->expanded) return;

std::wstring pattern = Utils::JoinPath(data->path, L"*");
WIN32_FIND_DATAW fd{};
HANDLE h = FindFirstFileW(pattern.c_str(), &fd);
if (h == INVALID_HANDLE_VALUE) return;
do {
    if ((fd.dwFileAttributes & FILE_ATTRIBUTE_DIRECTORY) == 0)
        continue;
    std::wstring name = fd.cFileName;
    if (name == L"." || name == L"..") continue;
    std::wstring full = Utils::JoinPath(data->path, name);
    bool hasChildren = HasSubdirs(full);
    InsertTreeItem(hTree, hItem, name, full, hasChildren);
} while (FindNextFileW(h, &fd));
FindClose(h);
data->expanded = true;
}

void FreeNodeDataRecursive(HWND hTree, HTREEITEM hItem) {
    while (hItem) {
        HTREEITEM child = TreeView_GetChild(hTree, hItem);
        if (child) FreeNodeDataRecursive(hTree, child);
        TVITEMW tvi{}; tvi.mask = TVIF_PARAM; tvi.hItem = hItem;
        TreeView_GetItem(hTree, &tvi);
        if (tvi.lParam) delete reinterpret_cast<NodeData*>(tvi.lParam);
        hItem = TreeView_GetNextSibling(hTree, hItem);
    }
}

HTREEITEM EnsurePathInTree(HWND hTree, const std::wstring &fullPath)
{
    if (fullPath.size() < 3 || fullPath[1] != L':') return nullptr;

    HTREEITEM hItem = TreeView_GetRoot(hTree);
    HTREEITEM driveItem = nullptr;
    while (hItem) {
        TVITEMW tvi{}; tvi.mask = TVIF_PARAM; tvi.hItem = hItem;
        TreeView_GetItem(hTree, &tvi);
        NodeData *data = reinterpret_cast<NodeData*>(tvi.lParam);
        if (data && _wcsicmp(data->path.c_str(), std::wstring(fullPath.substr(0,3)).c_str()) == 0) {
            driveItem = hItem; break;
        }
        hItem = TreeView_GetNextSibling(hTree, hItem);
    }
    if (!driveItem) return nullptr;

    if (fullPath.size() == 3) return driveItem;

    size_t pos = 3;

```

```

HTREEITEM current = driveItem;
while (pos < fullPath.size()) {
    size_t next = fullPath.find(L'\\', pos);
    std::wstring part = fullPath.substr(pos, next == std::
        wstring::npos ? std::wstring::npos : next - pos);
    if (part.empty()) break;

    ExpandTreeItem(hTree, current);

    HTREEITEM child = TreeView_GetChild(hTree, current);
    HTREEITEM foundChild = nullptr;
    while (child) {
        TVITEMW tv{}; tv.mask = TVIF_PARAM; tv.hItem = child;
        TreeView_GetItem(hTree, &tv);
        NodeData *cd = reinterpret_cast<NodeData*>(tv.lParam);
        if (cd) {
            const wchar_t *lastSep = wcsrchr(cd->path.c_str(), L
                '\\');
            std::wstring last = lastSep ? std::wstring(lastSep +
                1) : cd->path;
            if (_wcsicmp(last.c_str(), part.c_str()) == 0) {
                foundChild = child; break; }
        }
        child = TreeView_GetNextSibling(hTree, child);
    }
    if (!foundChild) return current;
    current = foundChild;
    if (next == std::wstring::npos) break;
    pos = next + 1;
}
return current;
}

void AddDriveRoots(HWND hTree) {
    DWORD mask = GetLogicalDrives();
    for (int i = 0; i < 26; ++i) {
        if (mask & (1u << i)) {
            wchar_t drive[4] = { wchar_t('A' + i), L':', L'\\', 0 };
            UINT driveType = GetDriveTypeW(drive);

            if (driveType == DRIVE_REMOTE) {
                continue;
            }

            std::wstring label = std::wstring(drive);
            bool hasChildren = HasSubdirs(label);
            InsertTreeItem(hTree, TVI_ROOT, label, label,
                hasChildren);
        }
    }
}

```

## Network

```

// Winsock must be included before windows.h
#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>
#include <commctrl.h>

```

```

#include <winnetwk.h>
#include <iphlpapi.h>
#include "../include/network.h"
#include "../include/filesystem.h"
#include <algorithm>

namespace Network {
    bool IsValidDeviceIP(const std::wstring& ip) {
        unsigned int b1, b2, b3, b4;
        if (swscanf(ip.c_str(), L"%u.%u.%u.%u", &b1, &b2, &b3, &b4) !=
            4) {
            return false;
        }

        if (b1 == 0 && b2 == 0 && b3 == 0 && b4 == 0) return false;
        if (b1 == 127) return false;
        if (b1 >= 224 && b1 <= 239) return false;
        if (b1 >= 240) return false;
        if (b1 == 255 && b2 == 255 && b3 == 255 && b4 == 255) return
            false;

        return true;
    }

    std::vector<std::wstring> GetNetworkIPsFromARP() {
        std::set<std::wstring> ipSet;
        std::set<std::wstring> localIPs;
        std::wstring defaultGateway;

        PMIB_IPADDRTABLE pIpAddrTable = nullptr;
        ULONG ulSize = 0;
        if (GetIpAddrTable(nullptr, &ulSize, FALSE) ==
            ERROR_INSUFFICIENT_BUFFER) {
            pIpAddrTable = (PMIB_IPADDRTABLE)malloc(ulSize);
            if (GetIpAddrTable(pIpAddrTable, &ulSize, FALSE) == NO_ERROR
                ) {
                for (DWORD i = 0; i < pIpAddrTable->dwNumEntries; i++) {
                    MIB_IPADDRROW* row = &pIpAddrTable->table[i];
                    if (row->dwAddr != 0 && row->dwAddr != 0x0100007F) {
                        IN_ADDR addr;
                        addr.S_un.S_addr = row->dwAddr;
                        wchar_t ipStr[16];
                        swprintf(ipStr, 16, L"%d.%d.%d.%d",
                            addr.S_un.S_un_b.s_b1,
                            addr.S_un.S_un_b.s_b2,
                            addr.S_un.S_un_b.s_b3,
                            addr.S_un.S_un_b.s_b4);
                        localIPs.insert(ipStr);
                    }
                }
            }
            free(pIpAddrTable);
        }

        PMIB_IPFORWARDTABLE pForwardTable = nullptr;
        ulSize = 0;
        if (GetIpForwardTable(nullptr, &ulSize, FALSE) ==
            ERROR_INSUFFICIENT_BUFFER) {
            pForwardTable = (PMIB_IPFORWARDTABLE)malloc(ulSize);

```

```

    if (GetIpForwardTable(pForwardTable, &ulSize, FALSE) ==
        NO_ERROR) {
        for (DWORD i = 0; i < pForwardTable->dwNumEntries; i++)
        {
            MIB_IPFORWARDROW* row = &pForwardTable->table[i];
            if (row->dwForwardDest == 0 && row->dwForwardMask ==
                0) {
                IN_ADDR addr;
                addr.S_un.S_addr = row->dwForwardNextHop;
                if (addr.S_un.S_addr != 0) {
                    wchar_t gatewayStr[16];
                    swprintf(gatewayStr, 16, L"%d.%d.%d.%d",
                        addr.S_un.S_un_b.s_b1,
                        addr.S_un.S_un_b.s_b2,
                        addr.S_un.S_un_b.s_b3,
                        addr.S_un.S_un_b.s_b4);
                    defaultGateway = gatewayStr;
                    break;
                }
            }
        }
        free(pForwardTable);
    }

    PMIB_IPNETTABLE pArpTable = nullptr;
    ulSize = 0;
    if (GetIpNetTable(nullptr, &ulSize, FALSE) ==
        ERROR_INSUFFICIENT_BUFFER) {
        pArpTable = (PMIB_IPNETTABLE)malloc(ulSize);
        if (GetIpNetTable(pArpTable, &ulSize, FALSE) == NO_ERROR) {
            for (DWORD i = 0; i < pArpTable->dwNumEntries; i++) {
                MIB_IPNETROW* row = &pArpTable->table[i];
                if (row->dwType == MIB_IPNET_TYPE_DYNAMIC || row->
                    dwType == MIB_IPNET_TYPE_STATIC) {
                    IN_ADDR addr;
                    addr.S_un.S_addr = row->dwAddr;
                    wchar_t ipStr[16];
                    swprintf(ipStr, 16, L"%d.%d.%d.%d",
                        addr.S_un.S_un_b.s_b1,
                        addr.S_un.S_un_b.s_b2,
                        addr.S_un.S_un_b.s_b3,
                        addr.S_un.S_un_b.s_b4);
                    std::wstring ip = ipStr;

                    if (IsValidDeviceIP(ip) && localIPs.find(ip) ==
                        localIPs.end()) {
                        ipSet.insert(ip);
                    }
                }
            }
        }
        free(pArpTable);
    }

    if (!defaultGateway.empty() &&
        IsValidDeviceIP(defaultGateway) &&
        localIPs.find(defaultGateway) == localIPs.end() &&
        ipSet.find(defaultGateway) == ipSet.end()) {
        ipSet.insert(defaultGateway);
    }

```

```

    }

    std::vector<std::wstring> devices;
    for (const auto& ip : ipSet) {
        devices.push_back(L"\\\\\\" + ip);
    }

    return devices;
}

void EnumerateNetworkResourcesRecursive(NETRESOURCEW* pNetResource,
                                       std::set<std::wstring>&
                                       devicesSet,
                                       int depth) {
    if (depth > 5) return; // Prevent infinite recursion

    HANDLE hEnum = nullptr;
    DWORD dwFlags = 0;
    // Use appropriate flags for enumeration
    if (pNetResource == nullptr) {
        // Root enumeration - find both containers and connectable
        // resources
        dwFlags = RESOURCEUSAGE_CONTAINER |
            RESOURCEUSAGE_CONNECTABLE;
    } else {
        // Use flags from the resource, but ensure we get containers
        dwFlags = 0; // Will use resource's flags
    }

    DWORD result = WNetOpenEnumW(RESOURCE_GLOBALNET,
                                RESOURCETYPE_ANY, dwFlags, pNetResource, &hEnum);

    if (result == NO_ERROR && hEnum) {
        DWORD bufferSize = 32768; // Larger buffer for more devices
        NETRESOURCEW* pBuffer = (NETRESOURCEW*)malloc(bufferSize);

        while (true) {
            DWORD count = 0xFFFFFFFF;
            result = WNetEnumResourceW(hEnum, &count, pBuffer, &
                                      bufferSize);

            if (result == NO_ERROR && count > 0) {
                for (DWORD i = 0; i < count; ++i) {
                    if (pBuffer[i].lpRemoteName) {
                        std::wstring remoteName = pBuffer[i].
                            lpRemoteName;

                        // Add servers (\\computername) to the set
                        if (remoteName.length() >= 2 &&
                            remoteName[0] == L'\\' && remoteName[1]
                                == L'\\') {
                            size_t thirdSlash = remoteName.find(L
                                '\\', 2);
                            if (thirdSlash == std::wstring::npos) {
                                // This is a server name (\\
                                    computername), add it
                                devicesSet.insert(remoteName);
                            } else {
                                // This is a share (\\computername\
                                    share), extract server name

```

```

        std::wstring serverName = remoteName
            .substr(0, thirdSlash);
        devicesSet.insert(serverName);
    }
}

// Recursively enumerate containers
// (workgroups, domains)
if (pBuffer[i].dwUsage &
    RESOURCEUSAGE_CONTAINER) {
    EnumerateNetworkResourcesRecursive(&
        pBuffer[i], devicesSet, depth + 1);
}
}

// Also check for connectable resources that
// might be servers
if ((pBuffer[i].dwUsage &
    RESOURCEUSAGE_CONNECTABLE) && pBuffer[i].
    lpRemoteName) {
    std::wstring remoteName = pBuffer[i].
        lpRemoteName;
    if (remoteName.length() >= 2 &&
        remoteName[0] == L'\\' && remoteName[1]
        == L'\\') {
        size_t thirdSlash = remoteName.find(L
            '\\', 2);
        if (thirdSlash == std::wstring::npos) {
            devicesSet.insert(remoteName);
        } else {
            std::wstring serverName = remoteName
                .substr(0, thirdSlash);
            devicesSet.insert(serverName);
        }
    }
}

}

} else if (result == ERROR_NO_MORE_ITEMS) {
    break;
} else {
    // Error occurred - may need larger buffer or
    // different approach
    // Try with larger buffer
    if (result == ERROR_MORE_DATA) {
        free(pBuffer);
        bufferSize *= 2;
        if (bufferSize > 65536) break; // Limit buffer
            size
        pBuffer = (NETRESOURCEW*)malloc(bufferSize);
        continue;
    }
    break;
}

}

free(pBuffer);
WNetCloseEnum(hEnum);
}

std::vector<std::wstring> EnumerateNetworkDrives() {

```



```

std::vector<std::wstring> devices;
DWORD mask = GetLogicalDrives();

for (int i = 0; i < 26; ++i) {
    if (mask & (1u << i)) {
        wchar_t drive[4] = { wchar_t('A' + i), L':', L'\\', 0 };
        UINT driveType = GetDriveTypeW(drive);

        if (driveType == DRIVE_REMOTE) {
            wchar_t remotePath[MAX_PATH + 1] = {0};
            DWORD bufSize = MAX_PATH;

            if (WNetGetConnectionW(drive, remotePath, &bufSize)
                == NO_ERROR) {
                devices.push_back(remotePath);
            } else {
                devices.push_back(drive);
            }
        }
    }
}

return devices;
}

std::vector<std::wstring> EnumerateUSBDevices() {
    std::vector<std::wstring> devices;
    DWORD mask = GetLogicalDrives();

    for (int i = 0; i < 26; ++i) {
        if (mask & (1u << i)) {
            wchar_t drive[4] = { wchar_t('A' + i), L':', L'\\', 0 };
            UINT driveType = GetDriveTypeW(drive);

            if (driveType == DRIVE_REMOVABLE || driveType ==
                DRIVE_CDROM || driveType == DRIVE_RAMDISK) {
                wchar_t volumeName[MAX_PATH + 1] = {0};
                wchar_t fileName[MAX_PATH + 1] = {0};
                DWORD serialNumber = 0;

                if (GetVolumeInformationW(drive, volumeName,
                    MAX_PATH, &serialNumber, nullptr, nullptr,
                    fileName, MAX_PATH)) {
                    devices.push_back(drive);
                } else {
                    devices.push_back(drive);
                }
            }
        }
    }

    return devices;
}

std::vector<std::wstring> EnumerateNetworkDevices() {
    std::set<std::wstring> devicesSet;
    std::vector<std::wstring> devices;

    // Method 1: Add network drives FIRST (mapped to drive letters
    // like Z:, Y:, etc.)

```

```

// These should always appear in the network view
std::vector<std::wstring> networkDrives =
    EnumerateNetworkDrives();
for (const auto& dev : networkDrives) {
    if (!dev.empty()) {
        devicesSet.insert(dev);
    }
}

// Method 2: Enumerate all network resources starting from root
// (like Windows Explorer Network view)
// This is the main method that finds network computers and
// shares
NETRESOURCEW* pNetResource = nullptr;
EnumerateNetworkResourcesRecursive(pNetResource, devicesSet, 0);

// Method 3: Enumerate workgroups/domains and their servers
NETRESOURCEW nr{};
nr.dwScope = RESOURCE_GLOBALNET;
nr.dwType = RESOURCETYPE_ANY;
nr.dwUsage = RESOURCEUSAGE_CONTAINER;
nr.dwDisplayType = RESOURCEDISPLAYTYPE_DOMAIN;
EnumerateNetworkResourcesRecursive(&nr, devicesSet, 0);

// Method 4: Enumerate servers directly
// (RESOURCEDISPLAYTYPE_SERVER)
nr.dwScope = RESOURCE_GLOBALNET;
nr.dwType = RESOURCETYPE_ANY;
nr.dwUsage = RESOURCEUSAGE_ALL;
nr.dwDisplayType = RESOURCEDISPLAYTYPE_SERVER;
EnumerateNetworkResourcesRecursive(&nr, devicesSet, 0);

// Method 5: Enumerate all disk shares (RESOURCETYPE_DISK)
nr.dwScope = RESOURCE_GLOBALNET;
nr.dwType = RESOURCETYPE_DISK;
nr.dwUsage = RESOURCEUSAGE_ALL;
nr.dwDisplayType = RESOURCEDISPLAYTYPE_GENERIC;
EnumerateNetworkResourcesRecursive(&nr, devicesSet, 0);

// Method 6: Try to enumerate from Network Neighborhood root
// This mimics what Windows Explorer shows in Network view
HANDLE hEnum = nullptr;
DWORD result = WNetOpenEnumW(RESOURCE_GLOBALNET,
    RESOURCETYPE_ANY,
    RESOURCEUSAGE_CONTAINER |
    RESOURCEUSAGE_CONNECTABLE,
    nullptr, &hEnum);
if (result == NO_ERROR && hEnum) {
    DWORD bufferSize = 16384;
    NETRESOURCEW* pBuffer = (NETRESOURCEW*)malloc(bufferSize);
    DWORD count = 0xFFFFFFFF;

    while (true) {
        count = 0xFFFFFFFF;
        result = WNetEnumResourceW(hEnum, &count, pBuffer, &
            bufferSize);
        if (result == NO_ERROR && count > 0) {
            for (DWORD i = 0; i < count; i++) {
                if (pBuffer[i].lpRemoteName) {

```

```

        std::wstring remoteName = pBuffer[i].
            lpRemoteName;
        // Add servers (\\computername)
        if (remoteName.length() >= 2 &&
            remoteName[0] == L'\\' && remoteName[1]
            == L'\\') {
            size_t thirdSlash = remoteName.find(L
                '\\', 2);
            if (thirdSlash == std::wstring::npos) {
                // It's a server, add it
                devicesSet.insert(remoteName);
            }
        }
        // Recursively enumerate containers
        if (pBuffer[i].dwUsage &
            RESOURCEUSAGE_CONTAINER) {
            EnumerateNetworkResourcesRecursive(&
                pBuffer[i], devicesSet, 0);
        }
    }
}
} else if (result == ERROR_NO_MORE_ITEMS) {
    break;
} else {
    break;
}
}
free(pBuffer);
WNetCloseEnum(hEnum);
}

// Method 7: Add devices from ARP table (IP addresses of devices
// in local network)
// This helps find devices that might not be in network
// enumeration
std::vector<std::wstring> arpDevices = GetNetworkIPsFromARP();
for (const auto& dev : arpDevices) {
    if (!dev.empty()) {
        devicesSet.insert(dev);
    }
}

// Convert set to vector
for (const auto& dev : devicesSet) {
    if (!dev.empty()) {
        devices.push_back(dev);
    }
}

return devices;
}

std::vector<std::wstring> EnumerateNetworkShares(const std::wstring&
    serverPath) {
    std::vector<std::wstring> shares;

    NETRESOURCEW nr{};
    nr.dwScope = RESOURCE_GLOBALNET;
    nr.dwType = RESOURCETYPE_DISK;
    nr.dwUsage = RESOURCEUSAGE_CONNECTABLE;

```

```

nr.dwDisplayType = RESOURCEDISPLAYTYPE_SHARE;
nr.lpRemoteName = const_cast<wchar_t*>(serverPath.c_str());

HANDLE hEnum = nullptr;
DWORD result = WNetOpenEnumW(RESOURCE_GLOBALNET,
    RESOURCETYPE_DISK, 0, &nr, &hEnum);

if (result == NO_ERROR && hEnum) {
    DWORD bufferSize = 16384;
    NETRESOURCEW* pBuffer = (NETRESOURCEW*)malloc(bufferSize);
    DWORD entries = 0xFFFFFFFF;

    while (true) {
        DWORD count = entries;
        result = WNetEnumResourceW(hEnum, &count, pBuffer, &
            bufferSize);

        if (result == NO_ERROR) {
            for (DWORD i = 0; i < count; ++i) {
                if (pBuffer[i].lpRemoteName) {
                    shares.push_back(pBuffer[i].lpRemoteName);
                }
            }
        } else if (result == ERROR_NO_MORE_ITEMS) {
            break;
        } else {
            break;
        }
    }

    free(pBuffer);
    WNetCloseEnum(hEnum);
}

return shares;
}

bool DeviceHasChildren(const std::wstring& device) {
    if (device.length() >= 2 && device[0] == L'\\' && device[1] == L'\\') {
        size_t thirdSlash = device.find(L'\\', 2);
        if (thirdSlash == std::wstring::npos) {
            std::vector<std::wstring> shares =
                EnumerateNetworkShares(device);
            return !shares.empty();
        } else {
            return Filesystem::HasSubdirs(device);
        }
    } else {
        NETRESOURCEW nr{};
        nr.dwScope = RESOURCE_GLOBALNET;
        nr.dwType = RESOURCETYPE_ANY;
        nr.dwUsage = RESOURCEUSAGE_CONTAINER;
        nr.lpRemoteName = const_cast<wchar_t*>(device.c_str());

        HANDLE hEnum = nullptr;
        DWORD enumResult = WNetOpenEnumW(RESOURCE_GLOBALNET,
            RESOURCETYPE_ANY, 0, &nr, &hEnum);
        if (enumResult == NO_ERROR && hEnum) {
            DWORD bufferSize = 16384;

```

```

        NETRESOURCEW* pBuffer = (NETRESOURCEW*)malloc(bufferSize
        );
        DWORD count = 1;
        enumResult = WNetEnumResourceW(hEnum, &count, pBuffer, &
        bufferSize);
        bool hasChildren = (enumResult == NO_ERROR && count > 0)
        ;
        free(pBuffer);
        WNetCloseEnum(hEnum);
        return hasChildren;
    }
}
return false;
}

void ExpandNetworkDevice(HWND hTree, HTREEITEM hItem) {
    HTREEITEM hChild = TreeView_GetChild(hTree, hItem);
    while (hChild) {
        TVITEMW tvi{}; tvi.mask = TVIF_TEXT; tvi.hItem = hChild;
        wchar_t buf[8]; tvi.pszText = buf; tvi.cchTextMax = 8;
        TreeView_GetItem(hTree, &tvi);
        if (wcscmp(buf, L"...") == 0) {
            TreeView_DeleteItem(hTree, hChild);
            break;
        }
        hChild = TreeView_GetNextSibling(hTree, hChild);
    }

    TVITEMW tvi{}; tvi.mask = TVIF_PARAM; tvi.hItem = hItem;
    TreeView_GetItem(hTree, &tvi);
    Filesystem::NodeData *data = reinterpret_cast<Filesystem::
    NodeData*>(tvi.lParam);
    if (!data || data->expanded) return;

    if (data->path.length() >= 2 && data->path[0] == L'\\' && data->
    path[1] == L'\\') {
        size_t thirdSlash = data->path.find(L'\\', 2);
        if (thirdSlash == std::wstring::npos) {
            std::vector<std::wstring> shares =
                EnumerateNetworkShares(data->path);

            for (const auto& share : shares) {
                std::wstring displayName = share;
                size_t lastSlash = displayName.find_last_of(L'\\');
                if (lastSlash != std::wstring::npos && lastSlash <
                displayName.length() - 1) {
                    displayName = displayName.substr(lastSlash + 1);
                }

                bool hasChildren = Filesystem::HasSubdirs(share);
                Filesystem::InsertTreeItem(hTree, hItem, displayName
                , share, hasChildren);
            }
            data->expanded = true;
            return;
        }
    }

    Filesystem::ExpandTreeItem(hTree, hItem);
}

```

```

void AddNetworkDevices(HWND hTree, const std::vector<std::wstring>&
    devices) {
    // Don't add anything if devices list is empty - let caller
    handle that
    if (devices.empty()) {
        return;
    }

    for (const auto& device : devices) {
        // Skip empty devices
        if (device.empty()) continue;

        std::wstring displayName = device;

        // Handle UNC paths (\\server\share)
        if (device.length() >= 2 && device[0] == L'\\' && device[1]
            == L'\\') {
            displayName = device.substr(2);
            // Remove trailing backslashes
            while (!displayName.empty() && displayName.back() == L
                '\\') {
                displayName.pop_back();
            }
            // If still empty after removing backslashes, use the
            original
            if (displayName.empty()) {
                displayName = device;
            }
        }
        // Handle drive letters (Z:, Y:, etc.) - network drives
        else if (device.length() >= 3 && device[1] == L':') {
            wchar_t volumeName[MAX_PATH + 1] = {0};
            if (GetVolumeInformationW(device.c_str(), volumeName,
                MAX_PATH, nullptr, nullptr, nullptr, nullptr, 0)) {
                if (wcslen(volumeName) > 0) {
                    displayName = std::wstring(device.substr(0, 2))
                        + L" (" + volumeName + L")";
                } else {
                    displayName = device.substr(0, 2);
                }
            } else {
                displayName = device.substr(0, 2);
            }
        }
    }

    if (displayName.empty()) continue;

    // Determine if device has children (shares or
    subdirectories)
    bool hasChildren = false;
    if (device.length() >= 2 && device[0] == L'\\' && device[1]
        == L'\\') {
        size_t thirdSlash = device.find(L'\\', 2);
        if (thirdSlash == std::wstring::npos) {
            // This is a server (\\server), check if it has
            shares
            hasChildren = DeviceHasChildren(device);
            // Always assume servers might have children, even
            if check fails

```

```

        if (!hasChildren) {
            hasChildren = true; // Allow expansion attempt
        }
    } else {
        // This is a share or subdirectory (\\server\share)
        hasChildren = DeviceHasChildren(device);
    }
} else if (device.length() >= 3 && device[1] == L':') {
    // Network drive - check if it has subdirectories
    hasChildren = Filesystem::HasSubdirs(device);
}

Filesystem::InsertTreeItem(hTree, TVI_ROOT, displayName,
    device, hasChildren);
}
}
}

```

## Settings

```

#include "../..//include/settings.h"
#include "../..//include/config.h"
#include <algorithm>

namespace Settings {
    // Global state
    static ThemeChoice g_themeChoice = ThemeChoice::System;
    static COLORREF g_colBg = RGB(255,255,255);
    static COLORREF g_colText = RGB(0,0,0);
    static HBRUSH g_hBgBrush = nullptr;
    static std::wstring g_fontFace = L"Segoe UI";
    static int g_fontPt = 12;
    static HFONT g_hFont = nullptr;

    ThemeChoice GetTheme() {
        return g_themeChoice;
    }

    void SetTheme(ThemeChoice theme) {
        g_themeChoice = theme;
    }

    COLORREF GetBackgroundColor() {
        return g_colBg;
    }

    COLORREF GetTextColor() {
        return g_colText;
    }

    HBRUSH GetBackgroundBrush() {
        return g_hBgBrush;
    }

    void SetBackgroundBrush(HBRUSH brush) {
        if (g_hBgBrush && g_hBgBrush != brush) {
            DeleteObject(g_hBgBrush);
        }
        g_hBgBrush = brush;
    }
}

```

```

bool QuerySystemAppsUseLightTheme() {
    DWORD value = 1;
    DWORD size = sizeof(value);
    LSTATUS st = RegGetValueW(HKEY_CURRENT_USER,
        L"Software\\Microsoft\\Windows\\CurrentVersion\\Themes\\
        Personalize",
        L"AppsUseLightTheme", RRF_RT_REG_DWORD, nullptr, &value, &
        size);
    if (st != ERROR_SUCCESS) return true; // default light if
        missing
    return value != 0;
}

void UpdateThemeColors() {
    ThemeChoice effective = g_themeChoice;
    if (effective == ThemeChoice::System) {
        effective = QuerySystemAppsUseLightTheme() ? ThemeChoice::
            Light : ThemeChoice::Dark;
    }
    if (effective == ThemeChoice::Dark) {
        g_colBg = RGB(25, 25, 25);
        g_colText = RGB(255, 255, 255);
    } else {
        g_colBg = RGB(255, 255, 255);
        g_colText = RGB(0, 0, 0);
    }
    if (g_hBgBrush) {
        DeleteObject(g_hBgBrush);
        g_hBgBrush = nullptr;
    }
    g_hBgBrush = CreateSolidBrush(g_colBg);
}

std::wstring GetFontFace() {
    return g_fontFace;
}

void SetFontFace(const std::wstring& face) {
    g_fontFace = face;
}

int GetFontSize() {
    return g_fontPt;
}

void SetFontSize(int size) {
    g_fontPt = size;
}

HFONT GetFont() {
    if (!g_hFont) {
        // Create font if it doesn't exist
        HDC hdc = GetDC(nullptr);
        int logPix = GetDeviceCaps(hdc, LOGPIXELSY);
        ReleaseDC(nullptr, hdc);
        int height = -MulDiv(g_fontPt, logPix, 72);
        g_hFont = CreateFontW(height, 0, 0, 0, FW_NORMAL, FALSE,
            FALSE, FALSE, DEFAULT_CHARSET,

```



```

        OUT_DEFAULT_PRECIS,
        CLIP_DEFAULT_PRECIS,
        CLEARTYPE_QUALITY,
        DEFAULT_PITCH | FF_DONTCARE,
        g_fontFace.c_str());
    }
    return g_hFont;
}

void SetFont(HFONT font) {
    if (g_hFont && g_hFont != font) {
        DeleteObject(g_hFont);
    }
    g_hFont = font;
}

void RecreateFont() {
    if (g_hFont) {
        DeleteObject(g_hFont);
        g_hFont = nullptr;
    }
    GetFont(); // This will create the font
}

void Save() {
    HKEY hKey;
    if (RegCreateKeyExW(HKEY_CURRENT_USER, L"Software\\KursWork\\
FileManager", 0, nullptr, 0, KEY_SET_VALUE, nullptr, &hKey,
nullptr) == ERROR_SUCCESS) {
        DWORD theme = (DWORD)g_themeChoice;
        RegSetValueExW(hKey, L"Theme", 0, REG_DWORD,
            reinterpret_cast<const BYTE*>(&theme), sizeof(theme));
        DWORD size = (DWORD)g_fontPt;
        RegSetValueExW(hKey, L"FontSize", 0, REG_DWORD,
            reinterpret_cast<const BYTE*>(&size), sizeof(size));
        RegSetValueExW(hKey, L"FontFace", 0, REG_SZ,
            reinterpret_cast<const BYTE*>(g_fontFace.c_str()),
            (DWORD)((g_fontFace.size() + 1) * sizeof(wchar_t)));
        RegCloseKey(hKey);
    }
}

void Load() {
    HKEY hKey;
    if (RegOpenKeyExW(HKEY_CURRENT_USER, L"Software\\KursWork\\
FileManager", 0, KEY_QUERY_VALUE, &hKey) == ERROR_SUCCESS) {
        DWORD theme = (DWORD)ThemeChoice::System;
        DWORD sz = sizeof(theme);
        if (RegGetValueW(hKey, nullptr, L"Theme", RRF_RT_REG_DWORD,
            nullptr, &theme, &sz) == ERROR_SUCCESS) {
            if (theme <= (DWORD)ThemeChoice::Dark) g_themeChoice =
                (ThemeChoice)theme;
        }
        DWORD size = 12;
        sz = sizeof(size);
        if (RegGetValueW(hKey, nullptr, L"FontSize",
            RRF_RT_REG_DWORD, nullptr, &size, &sz) == ERROR_SUCCESS)
        {
            if (size >= 8 && size <= 72) g_fontPt = (int)size;
        }
    }
}

```

```

        wchar_t buf[128];
        DWORD bytes = sizeof(buf);
        if (RegGetValueW(hKey, nullptr, L"FontFace", RRF_RT_REG_SZ,
            nullptr, buf, &bytes) == ERROR_SUCCESS) {
            g_fontFace = buf;
        }
        RegCloseKey(hKey);
    }
    UpdateThemeColors();
}
}

```

## UI

// Control initialization module

```

#ifndef UNICODE
#define UNICODE
#endif
#ifndef _UNICODE
#define _UNICODE
#endif

```

```

#include "../..//include/ui.h"
#include "../..//include/filesystem.h"
#include "../..//include/network.h"
#include "../..//include/settings.h"
#include "../..//include/config.h"
#include <windows.h>
#include <commctrl.h>
#include <shellapi.h>
#include <gdiplus.h>
#include <thread>
#include <mutex>

```

using namespace Gdiplus;

// Forward declarations

```

extern WindowData* GetWindowData(HWND hwnd);
extern void SetWindowDataInternal(HWND hwnd, WindowData* data);
extern LRESULT CALLBACK BorderSubclassProc(HWND hwnd, UINT uMsg, WPARAM
    wParam, LPARAM lParam, UINT_PTR uIdSubclass, DWORD_PTR dwRefData);
extern LRESULT CALLBACK ListViewSubclassProc(HWND hwnd, UINT uMsg,
    WPARAM wParam, LPARAM lParam);

```

```

namespace UI {
    void AddTooltipFor(HWND hParent, HWND hCtrl, const wchar_t *text) {
        HWND hToolTip = GetToolTip(hParent);
        if (!hToolTip) {
            hToolTip = CreateWindowExW(WS_EX_TOPMOST, TOOLTIPS_CLASSW,
                nullptr,
                WS_POPUP | TTS_NOPREFIX | TTS_ALWAYSTIP,
                CW_USEDEFAULT, CW_USEDEFAULT, CW_USEDEFAULT,
                CW_USEDEFAULT,
                hParent, nullptr, GetModuleHandleW(nullptr), nullptr);
            SetWindowPos(hToolTip, HWND_TOPMOST, 0, 0, 0, 0,
                SWP_NOMOVE | SWP_NOSIZE | SWP_NOACTIVATE);
            SendMessageW(hToolTip, TTM_SETMAXTIPWIDTH, 0, 300);
            SetToolTip(hParent, hToolTip);
        }
    }
}

```

```

    TOOLINFOW ti{}; ti.cbSize = sizeof(ti);
    ti.uFlags = TTF_SUBCLASS | TTF_IDISHWND;
    ti.hwnd = hParent;
    ti.uId = (UINT_PTR)hCtrl;
    ti.lpszText = const_cast<wchar_t*>(text);
    GetClientRect(hCtrl, &ti.rect);
    SendMessageW(hToolTip, TTM_ADDTOOL, 0, (LPARAM)&ti);
}

HICON LoadIconFromPNG(const wchar_t *path, ULONG_PTR gdiplusToken) {
    HICON hIcon = nullptr;
    Image *image = new Image(path);
    if (image && image->GetLastStatus() == Ok) {
        int smallIconSize = GetSystemMetrics(SM_CXSMICON);
        int largeIconSize = GetSystemMetrics(SM_CXICON);
        int iconSize = std::max(largeIconSize, 32);
        if (iconSize < 16) iconSize = 32;
        if (iconSize > 256) iconSize = 256;
        Bitmap *bitmap = new Bitmap(iconSize, iconSize);
        Graphics *graphics = Graphics::FromImage(bitmap);
        graphics->
            SetInterpolationMode(InterpolationModeHighQualityBicubic
            );
        graphics->SetSmoothingMode(SmoothingModeHighQuality);
        graphics->SetPixelOffsetMode(PixelOffsetModeHighQuality);
        graphics->DrawImage(image, 0, 0, iconSize, iconSize);
        bitmap->GetHICON(&hIcon);
        delete graphics;
        delete bitmap;
        delete image;
    } else {
        if (image) delete image;
    }
    return hIcon;
}

void InitializeControls(HWND hwnd, UIControls &controls, UIState &
state) {
    INITCOMMONCONTROLSEX icc{ sizeof(icc), ICC_TREEVIEW_CLASSES |
        ICC_LISTVIEW_CLASSES };
    InitCommonControlsEx(&icc);

    UI::BuildMenu(hwnd);
    UI::BuildAccelerators(hwnd, controls.hAccel);
    Settings::Load();

    // Create and store WindowData
    WindowData* wndData = UI::CreateWindowData();
    UI::InitializeWindowData(wndData, controls, state);
    SetWindowDataInternal(hwnd, wndData);

    // Create buttons with Unicode arrow symbols
    controls.hBackBtn = CreateWindowExW(0, L"BUTTON", L"←",
        WS_CHILD | WS_VISIBLE |
        WS_TABSTOP |
        BS_PUSHBUTTON,
        0, 0, 28, 28, hwnd, (HMENU)
        ID_BTN_BACK,
        GetModuleHandleW(nullptr),
        nullptr);
}

```

```

controls.hFwdBtn = CreateWindowExW(0, L"BUTTON", L"",
                                   WS_CHILD | WS_VISIBLE |
                                   WS_TABSTOP |
                                   BS_PUSHBUTTON,
                                   0, 0, 28, 28, hwnd, (HMENU)
                                   ID_BTN_FWD,
                                   GetModuleHandleW(nullptr),
                                   nullptr);

SetWindowTextW(controls.hFwdBtn, L"");
controls.hViewDetailsBtn = CreateWindowExW(0, L"BUTTON", L"",
                                             WS_CHILD | WS_VISIBLE |
                                             WS_TABSTOP |
                                             BS_PUSHBUTTON,
                                             0, 0, 28, 28, hwnd, (HMENU)
                                             ID_BTN_VIEW_DETAILS,
                                             GetModuleHandleW(nullptr),
                                             nullptr);

controls.hViewLargeBtn = CreateWindowExW(0, L"BUTTON", L"",
                                           WS_CHILD | WS_VISIBLE |
                                           WS_TABSTOP |
                                           BS_PUSHBUTTON,
                                           0, 0, 28, 28, hwnd, (HMENU)
                                           ID_BTN_VIEW_LARGE,
                                           GetModuleHandleW(nullptr),
                                           nullptr);

controls.hViewSmallBtn = CreateWindowExW(0, L"BUTTON", L"",
                                           WS_CHILD | WS_VISIBLE |
                                           WS_TABSTOP |
                                           BS_PUSHBUTTON,
                                           0, 0, 28, 28, hwnd, (HMENU)
                                           ID_BTN_VIEW_SMALL,
                                           GetModuleHandleW(nullptr),
                                           nullptr);

// Create path edit field
controls.hPathEdit = CreateWindowExW(0, L"EDIT", L"",
                                       WS_CHILD | WS_VISIBLE |
                                       WS_TABSTOP |
                                       ES_AUTOHSCROLL | WS_BORDER,
                                       0, 0, 200, 24, hwnd, (HMENU)
                                       ID_EDIT_PATH,
                                       GetModuleHandleW(nullptr),
                                       nullptr);

SetWindowSubclass(controls.hPathEdit, BorderSubclassProc, 1, 0);

// Create Go button
controls.hGoBtn = CreateWindowExW(0, L"BUTTON", L"Перейти",
                                   WS_CHILD | WS_VISIBLE | WS_TABSTOP,
                                   0, 0, 60, 24, hwnd, (HMENU)
                                   ID_BTN_GO,
                                   GetModuleHandleW(nullptr),
                                   nullptr);

// Create trees
controls.hTree = CreateWindowExW(0, WC_TREEVIEW, L"",
                                  WS_CHILD | WS_VISIBLE |
                                  TVS_HASLINES | TVS_LINESATROOT
                                  | TVS_HASBUTTONS | WS_BORDER,

```

```

0, 0, 100, 100, hwnd, (HMENU)
    ID_TREE_TOP,
    GetModuleHandleW(nullptr),
    nullptr);
SetWindowSubclass(controls.hTree, BorderSubclassProc, 1, 0);

// Create splitter
controls.hSplitter = CreateWindowExW(0, L"STATIC", L"",
    WS_CHILD | WS_VISIBLE,
    0, 0, 100, 4, hwnd, (HMENU)
    ID_SPLITTER,
    GetModuleHandleW(nullptr),
    nullptr);

// Create bottom tree for network devices
controls.hTreeBottom = CreateWindowExW(0, WC_TREEVIEW, L"",
    WS_CHILD | WS_VISIBLE |
    TVS_HASLINES |
    TVS_LINESATROOT |
    TVS_HASBUTTONS |
    WS_BORDER,
    0, 0, 100, 100, hwnd, (HMENU)
    ID_TREE_BOTTOM,
    GetModuleHandleW(nullptr),
    nullptr);
SetWindowSubclass(controls.hTreeBottom, BorderSubclassProc, 1,
    0);

// Create list view
controls.hList = CreateWindowExW(0, WC_LISTVIEW, L"",
    WS_CHILD | WS_VISIBLE | LVS_REPORT
    | LVS_SINGLESEL |
    LVS_SHAREIMAGELISTS |
    WS_BORDER,
    0, 0, 100, 100, hwnd, (HMENU)
    ID_LIST,
    GetModuleHandleW(nullptr),
    nullptr);
SetWindowSubclass(controls.hList, BorderSubclassProc, 1, 0);

// Attach system imagelists for icons
{
    SHFILEINFOW sfi{};
    HIMAGELIST hSmall = (HIMAGELIST)SHGetFileInfoW(L"C:\\",
        FILE_ATTRIBUTE_DIRECTORY, &sfi, sizeof(sfi),
        SHGFI_SYSICONINDEX | SHGFI_SMALLICON);
    HIMAGELIST hLarge = (HIMAGELIST)SHGetFileInfoW(L"C:\\",
        FILE_ATTRIBUTE_DIRECTORY, &sfi, sizeof(sfi),
        SHGFI_SYSICONINDEX | SHGFI_LARGEICON);
    if (hSmall) ListView_SetImageList(controls.hList, hSmall,
        LVSIL_SMALL);
    if (hLarge) ListView_SetImageList(controls.hList, hLarge,
        LVSIL_NORMAL);
}

// Set up list view columns
SetListColumns(controls.hList);
AdjustReportColumns(controls.hList);
DWORD ex = ListView_GetExtendedListViewStyle(controls.hList);
ex |= LVS_EX_FULLROWSELECT | LVS_EX_LABELTIP;

```

```

ThemeChoice effective = Settings::GetTheme();
if (effective == ThemeChoice::System) {
    effective = Settings::QuerySystemAppsUseLightTheme() ?
        ThemeChoice::Light : ThemeChoice::Dark;
}
if (effective != ThemeChoice::Dark) {
    ex |= LVS_EX_GRIDLINES;
}
ListView_SetExtendedListViewStyle(controls.hList, ex);

// Subclass ListView to draw empty message
state.oldListProc = (WNDPROC)SetWindowLongPtrW(controls.hList,
    GWLP_WNDPROC, (LONG_PTR)ListViewSubclassProc);

// Create empty label
controls.hEmptyLabel = CreateWindowExW(0, L"STATIC", L"Выберите"
    "директорию",
    WS_CHILD | SS_CENTER |
    SS_CENTERIMAGE,
    0, 0, 100, 100, hwnd, nullptr,
    GetModuleHandleW(nullptr),
    nullptr);
ShowWindow(controls.hEmptyLabel, SW_HIDE);

// Create status bar
controls.hStatusBar = CreateWindowExW(0, STATUSCLASSNAMEW, L"",
    WS_CHILD | WS_VISIBLE |
    SBARS_SIZEGRIP,
    0, 0, 0, 0, hwnd, (HMENU)
    ID_STATUS_BAR,
    GetModuleHandleW(nullptr),
    nullptr);

int parts[] = { -1 };
SendMessageW(controls.hStatusBar, SB_SETPARTS, 1, (LPARAM)parts)
    ;

AdjustReportColumns(controls.hList);
UpdateListViewRowHeight(controls.hList);
Filesystem::AddDriveRoots(controls.hTree);

// Clear bottom tree and add loading placeholder
TreeView_DeleteAllItems(controls.hTreeBottom);
state.loadingItem = Filesystem::InsertTreeItem(controls.
    hTreeBottom, TVI_ROOT, L"Загрузка...", L"", false);
UI::UpdateWindowState(hwnd, state);

LayoutChildren(hwnd, controls, state);

// Start network scan in background thread
state.networkScanInProgress = true;
UI::UpdateWindowState(hwnd, state);
std::thread([hwnd]() {
    std::vector<std::wstring> devices;
    try {
        Sleep(100);
        devices = Network::EnumerateNetworkDevices();
    } catch (const std::exception& e) {
        devices.clear();
    } catch (...) {

```

```

        devices.clear();
    }

    UI::UpdateScannedDevices(hwnd, devices);
    PostMessageW(hwnd, WM_USER_NETWORK_SCAN_COMPLETE, 0, 0);
}).detach();

UI::UpdateWindowData(hwnd, controls, state);

EnableWindow(controls.hBackBtn, FALSE);
EnableWindow(controls.hFwdBtn, FALSE);
UI::ApplyThemeToControls(hwnd, controls);
UI::ApplyFontToControls(hwnd, controls);
UI::UpdateMenuChecks(hwnd);

if (state.currentPath.empty()) {
    ShowMessage(controls.hList, controls.hEmptyLabel, L"Выберите"
        директорию");
}

UI::AddTooltipFor(hwnd, controls.hBackBtn, L"Назад"\nAlt+Left");
UI::AddTooltipFor(hwnd, controls.hFwdBtn, L"Вперёд"\nAlt+Right");
UI::AddTooltipFor(hwnd, controls.hViewDetailsBtn, L"Вид":
    Подробно");
UI::AddTooltipFor(hwnd, controls.hViewLargeBtn, L"Вид":
    Крупные значки ");
UI::AddTooltipFor(hwnd, controls.hViewSmallBtn, L"Вид":
    Мелкие значки ");
if (controls.hGoBtn) UI::AddTooltipFor(hwnd, controls.hGoBtn, L
    Перейти" к указанному пути ");
}
}

// Layout management module

#ifndef UNICODE
#define UNICODE
#endif
#ifndef _UNICODE
#define _UNICODE
#endif

#include "../include/ui.h"
#include "../include/settings.h"
#include <windows.h>

namespace UI {
    void LayoutChildren(HWND hwnd, UIControls &controls, UIState &state)
    {
        RECT rc; GetClientRect(hwnd, &rc);
        int width = rc.right - rc.left;
        int height = rc.bottom - rc.top;
        int leftW = width / 3;
        int rightX = leftW;

        // Calculate sizes based on font size for adaptive layout
        int fontSize = Settings::GetFontSize();
        HDC hdc = GetDC(hwnd);
        int logPix = GetDeviceCaps(hdc, LOGPIXELSY);
        ReleaseDC(hwnd, hdc);
    }
}

```

```

// Base size scales with font (12pt = 32px control bar, scale
// proportionally)
int ctlBarH = MulDiv(fontSize * 8, logPix, 72 * 3); //
    Approximately fontSize * 2.67
if (ctlBarH < 24) ctlBarH = 24; // Minimum height
if (ctlBarH > 60) ctlBarH = 60; // Maximum height

int pad = fontSize / 2; // Padding scales with font
if (pad < 4) pad = 4;
if (pad > 12) pad = 12;

int btnH = ctlBarH - pad - pad/2;
int btnW = btnH; // Square buttons
int x = rightX + pad;
int splitterH = 4;

int minTreeH = 50;
if (state.splitterPos < minTreeH) state.splitterPos = minTreeH;
if (state.splitterPos > height - minTreeH - splitterH) state.
    splitterPos = height - minTreeH - splitterH;

int topTreeH = state.splitterPos;
MoveWindow(controls.hTree, 0, 0, leftW, topTreeH, TRUE);
MoveWindow(controls.hSplitter, 0, topTreeH, leftW, splitterH,
    TRUE);

int bottomTreeY = topTreeH + splitterH;
int bottomTreeH = height - bottomTreeY;
MoveWindow(controls.hTreeBottom, 0, bottomTreeY, leftW,
    bottomTreeH, TRUE);

MoveWindow(controls.hBackBtn, x, pad, btnW, btnH, TRUE); x +=
    btnW + pad;
MoveWindow(controls.hFwdBtn, x, pad, btnW, btnH, TRUE); x +=
    btnW + pad*2;
MoveWindow(controls.hViewDetailsBtn, x, pad, btnW, btnH, TRUE);
    x += btnW + pad;
MoveWindow(controls.hViewLargeBtn, x, pad, btnW, btnH, TRUE); x
    += btnW + pad;
MoveWindow(controls.hViewSmallBtn, x, pad, btnW, btnH, TRUE); x
    += btnW + pad*2;

int pathEditX = x;
// Go button width scales with font
int goBtnW = MulDiv(fontSize * 5, logPix, 72); // Approximately
    fontSize * 0.7
if (goBtnW < 50) goBtnW = 50;
if (goBtnW > 100) goBtnW = 100;
int pathEditW = width - pathEditX - goBtnW - pad;
if (controls.hPathEdit && controls.hGoBtn) {
    if (pathEditW > 100) {
        ShowWindow(controls.hPathEdit, SW_SHOW);
        MoveWindow(controls.hPathEdit, pathEditX, pad, pathEditW
            , btnH, TRUE);
        MoveWindow(controls.hGoBtn, pathEditX + pathEditW, pad,
            goBtnW, btnH, TRUE);
    } else {
        ShowWindow(controls.hPathEdit, SW_HIDE);
        MoveWindow(controls.hGoBtn, pathEditX, pad, goBtnW, btnH
            , TRUE);
    }
}

```



```

    }
}

int listX = rightX;
int listY = ctlBarH;
int listW = width - rightX;
int listH = height - ctlBarH;
MoveWindow(controls.hList, listX, listY, listW, listH, TRUE);
if (controls.hEmptyLabel) {
    MoveWindow(controls.hEmptyLabel, listX, listY, listW, listH,
        TRUE);
}
if (controls.hStatusBar) {
    RECT statusRect;
    GetClientRect(controls.hStatusBar, &statusRect);
    MoveWindow(controls.hStatusBar, listX, height - statusRect.
        bottom, listW, statusRect.bottom, TRUE);
}
}

// List view operations module

#ifndef UNICODE
#define UNICODE
#endif
#ifndef _UNICODE
#define _UNICODE
#endif

#include "../include/ui.h"
#include "../include/settings.h"
#include <windows.h>
#include <commctrl.h>
#include <shellapi.h>

namespace UI {
    // Helper function to get icon size based on font size
    int GetIconSize() {
        int fontSize = Settings::GetFontSize();
        HDC hdc = GetDC(nullptr);
        int logPix = GetDeviceCaps(hdc, LOGPIXELSY);
        ReleaseDC(nullptr, hdc);
        // Icon size should be roughly 1.5x font height for good
        // visibility
        int iconSize = MulDiv(fontSize * 3, logPix, 72 * 2); // fontSize
            * 1.5
        // Clamp to reasonable sizes (16-48 pixels)
        if (iconSize < 16) iconSize = 16;
        if (iconSize > 48) iconSize = 48;
        return iconSize;
    }

    // Update list view row height to accommodate proportional icon size
    void UpdateListViewRowHeight(HWND hList) {
        int fontSize = Settings::GetFontSize();
        HDC hdc = GetDC(nullptr);
        int logPix = GetDeviceCaps(hdc, LOGPIXELSY);
        ReleaseDC(nullptr, hdc);

```

```

        // Row height should be proportional to font size (1.8x font
        // height for icon + text)
        int rowHeight = MulDiv(fontSize * 9, logPix, 72 * 5); //
        //      fontSize * 1.8
        // Clamp to reasonable sizes
        if (rowHeight < 20) rowHeight = 20;
        if (rowHeight > 60) rowHeight = 60;
        // Use LVM_SETITEMHEIGHT message directly
        #ifndef LVM_FIRST
        #define LVM_FIRST 0x1000
        #endif
        #ifndef LVM_SETITEMHEIGHT
        #define LVM_SETITEMHEIGHT (LVM_FIRST + 141)
        #endif
        SendMessageW(hList, LVM_SETITEMHEIGHT, 0, (LPARAM) rowHeight);
    }

void SetListColumns(HWND hList) {
    // Set extended style - gridlines will be drawn manually in dark
    // theme
    ThemeChoice effective = Settings::GetTheme();
    if (effective == ThemeChoice::System) {
        effective = Settings::QuerySystemAppsUseLightTheme() ?
            ThemeChoice::Light : ThemeChoice::Dark;
    }
    DWORD exStyle = LVS_EX_FULLROWSELECT | LVS_EX_LABELTIP;
    if (effective != ThemeChoice::Dark) {
        exStyle |= LVS_EX_GRIDLINES; // Only use system gridlines in
        // light theme
    }
    ListView_SetExtendedListViewStyle(hList, exStyle);

    LVCOLUMNW col{};
    col.mask = LVCF_TEXT | LVCF_WIDTH | LVCF_SUBITEM | LVCF_FMT;
    col.fmt = LVCFMT_LEFT;
    wchar_t nameCol[] = L"Name";
    wchar_t typeCol[] = L"Type";
    wchar_t modCol[] = L"Modified";
    col.pszText = nameCol;
    col.cx = 360; col.iSubItem = 0;
    ListView_InsertColumn(hList, 0, &col);
    col.pszText = typeCol;
    col.cx = 120; col.iSubItem = 1;
    ListView_InsertColumn(hList, 1, &col);
    col.pszText = modCol;
    col.cx = 200; col.iSubItem = 2;
    ListView_InsertColumn(hList, 2, &col);
}

void AdjustReportColumns(HWND hList) {
    LONG_PTR style = GetWindowLongPtrW(hList, GWL_STYLE);
    if ((style & LVS_TYPEMASK) != LVS_REPORT) return;
    RECT rc{}; GetClientRect(hList, &rc);
    int clientW = rc.right - rc.left;
    if (GetWindowLongPtrW(hList, GWL_STYLE) & WS_VSCROLL) {
        clientW -= GetSystemMetrics(SM_CXVSCROLL);
    }
    if (clientW <= 0) return;
    int typeW = 120;
    int modW = 200;

```

```

int minCol = 60;
int nameW = clientW - typeW - modW;
if (nameW < minCol) {
    int deficit = minCol - nameW;
    nameW = minCol;
    int reduceType = std::min(deficit / 2, std::max(0, typeW - minCol));
    typeW -= reduceType;
    deficit -= reduceType;
    int reduceMod = std::min(deficit, std::max(0, modW - minCol));
    modW -= reduceMod;
    deficit -= reduceMod;
    if (deficit > 0) nameW = std::max(minCol, nameW - deficit);
}
ListView_SetColumnWidth(hList, 1, typeW);
ListView_SetColumnWidth(hList, 2, modW);
ListView_SetColumnWidth(hList, 0, std::max(1, clientW - typeW - modW));
ShowScrollBar(hList, SB_HORZ, FALSE);
}

void SetListViewMode(HWND hList, DWORD mode) {
    LONG_PTR style = GetWindowLongPtrW(hList, GWL_STYLE);
    style &= ~(LVS_TYPEMASK);
    style |= mode;
    SetWindowLongPtrW(hList, GWL_STYLE, style);
    if ((mode & LVS_TYPEMASK) == LVS_REPORT) {
        while (ListView_DeleteColumn(hList, 0)) {}
        SetListColumns(hList);
        DWORD ex = ListView_GetExtendedListViewStyle(hList);
        ex |= LVS_EX_FULLROWSELECT | LVS_EX_LABELTIP;
        ListView_SetExtendedListViewStyle(hList, ex);
        // Ensure image lists are set for report mode too (icons in first column)
        SHFILEINFOW sfi{};
        HIMAGELIST hSmall = (HIMAGELIST)SHGetFileInfoW(L"C:\\",
            FILE_ATTRIBUTE_DIRECTORY, &sfi, sizeof(sfi),
            SHGFI_SYSICONINDEX | SHGFI_SMALLICON);
        if (hSmall) {
            ListView_SetImageList(hList, hSmall, LVSIL_SMALL);
        }
    }
    // Ensure image lists are set for icon modes
    if ((mode & LVS_TYPEMASK) == LVS_ICON || (mode & LVS_TYPEMASK) == LVS_SMALLICON) {
        SHFILEINFOW sfi{};
        HIMAGELIST hSmall = (HIMAGELIST)SHGetFileInfoW(L"C:\\",
            FILE_ATTRIBUTE_DIRECTORY, &sfi, sizeof(sfi),
            SHGFI_SYSICONINDEX | SHGFI_SMALLICON);
        HIMAGELIST hLarge = (HIMAGELIST)SHGetFileInfoW(L"C:\\",
            FILE_ATTRIBUTE_DIRECTORY, &sfi, sizeof(sfi),
            SHGFI_SYSICONINDEX | SHGFI_LARGEICON);
        if (hSmall) ListView_SetImageList(hList, hSmall, LVSIL_SMALL);
        if (hLarge) ListView_SetImageList(hList, hLarge, LVSIL_NORMAL);
    }
    InvalidateRect(hList, nullptr, TRUE);
}

```

```

bool IsReportMode(HWND hList) {
    LONG_PTR style = GetWindowLongPtrW(hList, GWL_STYLE);
    return (style & LVS_TYPEMASK) == LVS_REPORT;
}

bool IsIconMode(HWND hList) {
    LONG_PTR style = GetWindowLongPtrW(hList, GWL_STYLE);
    DWORD t = (DWORD)(style & LVS_TYPEMASK);
    return t == LVS_ICON || t == LVS_SMALLICON;
}

void AdjustIconLayout(HWND hList) {
    if (!IsIconMode(hList)) return;
    RECT rc{}; GetClientRect(hList, &rc);
    int clientW = rc.right - rc.left;
    if (clientW <= 0) return;
    LONG_PTR style = GetWindowLongPtrW(hList, GWL_STYLE);
    BOOL isSmall = ((style & LVS_TYPEMASK) == LVS_SMALLICON);
    if (isSmall) {
        style &= ~LVS_ALIGNLEFT;
        style |= LVS_AUTOARRANGE | LVS_ALIGNTOP;
    } else {
        style &= ~LVS_ALIGNTOP;
        style |= LVS_AUTOARRANGE | LVS_ALIGNLEFT;
    }
    SetWindowLongPtrW(hList, GWL_STYLE, style);
    LRESULT spacing = SendMessageW(hList, LVM_GETITEMSPACING,
        (WPARAM)isSmall, 0);
    int defX = LOWORD(spacing);
    int defY = HIWORD(spacing);
    if (defX <= 0) defX = 100;
    int cols = std::max(1, clientW / defX);
    int newX = std::max(32, clientW / cols);
    SendMessageW(hList, LVM_SETICONSPACING, 0, MAKELPARAM(newX, defY));
    ListView_Arrange(hList, LVA_DEFAULT);
    ShowScrollBar(hList, SB_HORZ, FALSE);
}

void ClearList(HWND hList) {
    ListView_DeleteAllItems(hList);
}

void ShowMessage(HWND hList, HWND hEmptyLabel, const std::wstring &
    message) {
    ClearList(hList);

    // Hide list view header when showing message
    HWND hHeader = ListView_GetHeader(hList);
    if (hHeader) {
        ShowWindow(hHeader, SW_HIDE);
    }

    if (hEmptyLabel) {
        SetWindowTextW(hEmptyLabel, message.c_str());
        ShowWindow(hEmptyLabel, SW_SHOW);
        SetWindowPos(hEmptyLabel, HWND_TOP, 0, 0, 0, 0, SWP_NOMOVE |
            SWP_NOSIZE);
        InvalidateRect(hEmptyLabel, nullptr, TRUE);
    }
}

```

```

    }
    ShowWindow(hList, SW_HIDE);
    InvalidateRect(hList, nullptr, TRUE);
    UpdateWindow(hList);
    RedrawWindow(hList, nullptr, nullptr, RDW_INVALIDATE |
        RDW_UPDATENOW | RDW_ERASE);
}

void UpdateStatusBar(HWND hStatusBar, const std::vector<FileSystem::
    ListItem> &items) {
    if (!hStatusBar) return;

    int fileCount = 0;
    int dirCount = 0;
    __int64 totalSize = 0;

    for (const auto& item : items) {
        if (item.isDir) {
            dirCount++;
        } else {
            fileCount++;
            WIN32_FIND_DATA fd{};
            HANDLE h = FindFirstFileW(item.fullPath.c_str(), &fd);
            if (h != INVALID_HANDLE_VALUE) {
                totalSize += ((__int64)fd.nFileSizeHigh << 32) | fd.
                    nFileSizeLow;
                FindClose(h);
            }
        }
    }

    wchar_t statusText[256];
    if (fileCount == 0 && dirCount == 0) {
        swprintf(statusText, 256, L"Готово");
    } else {
        swprintf(statusText, 256, L"Файлов": %d, Папок: %d",
            fileCount, dirCount);
    }
    SendMessageW(hStatusBar, SB_SETTEXT, 0, (LPARAM)statusText);
}

// Menu and accelerators module

#ifndef UNICODE
#define UNICODE
#endif
#ifndef _UNICODE
#define _UNICODE
#endif

#include "../include/ui.h"
#include "../include/settings.h"
#include "../include/config.h"
#include <windows.h>

namespace UI {
    void BuildMenu(HWND hwnd) {
        HMENU hBar = CreateMenu();
        HMENU hFile = CreatePopupMenu();
    }
}

```

```

HMENU hSettings = CreatePopupMenu();
HMENU hTheme = CreatePopupMenu();
HMENU hFont = CreatePopupMenu();
HMENU hFontSize = CreatePopupMenu();

AppendMenuW(hFile, MF_STRING, ID_MENU_FILE_ABOUT, LO" программе\
tF1");
AppendMenuW(hFile, MF_SEPARATOR, 0, nullptr);
AppendMenuW(hFile, MF_STRING, ID_MENU_FILE_EXIT, LVыйти"\tCtrl+Q
");

AppendMenuW(hTheme, MF_STRING | MF_CHECKED, ID_MENU_THEME_SYSTEM
, LCистемная"\tCtrl+1");
AppendMenuW(hTheme, MF_STRING, ID_MENU_THEME_LIGHT, LCветлая"\
tCtrl+2");
AppendMenuW(hTheme, MF_STRING, ID_MENU_THEME_DARK, LTёмная"\
tCtrl+3");

AppendMenuW(hFont, MF_STRING | MF_CHECKED, ID_MENU_FONT_SEGOE, L
"Segoe UI\tCtrl+4");
AppendMenuW(hFont, MF_STRING, ID_MENU_FONT_CONSOLAS, L"Consolas\
tCtrl+5");
AppendMenuW(hFont, MF_STRING, ID_MENU_FONT_COURIER, L"Courier
New\tCtrl+6");

AppendMenuW(hFontSize, MF_STRING, ID_MENU_FONTSIZE_10, L"10\
tCtrl+Shift+1");
AppendMenuW(hFontSize, MF_STRING | MF_CHECKED,
ID_MENU_FONTSIZE_12, L"12\tCtrl+Shift+2");
AppendMenuW(hFontSize, MF_STRING, ID_MENU_FONTSIZE_14, L"14\
tCtrl+Shift+3");
AppendMenuW(hFontSize, MF_STRING, ID_MENU_FONTSIZE_16, L"16\
tCtrl+Shift+4");

AppendMenuW(hSettings, MF_POPUP, (UINT_PTR)hTheme, LTема""");
AppendMenuW(hSettings, MF_POPUP, (UINT_PTR)hFont, LШрифт""");
AppendMenuW(hSettings, MF_POPUP, (UINT_PTR)hFontSize, LРазмер"
текста");

AppendMenuW(hBar, MF_POPUP, (UINT_PTR)hFile, LФайл""");
AppendMenuW(hBar, MF_POPUP, (UINT_PTR)hSettings, LНастройки""");

SetMenu(hwnd, hBar);
}

void BuildAccelerators(HWND hwnd, HACCEL &hAccel) {
ACCEL acc[] = {
    { FVIRTKEY, VK_F1, ID_MENU_FILE_ABOUT },
    { FVIRTKEY | FCONTROL, 'Q', ID_MENU_FILE_EXIT },
    { FVIRTKEY | FCONTROL, '1', ID_MENU_THEME_SYSTEM },
    { FVIRTKEY | FCONTROL, '2', ID_MENU_THEME_LIGHT },
    { FVIRTKEY | FCONTROL, '3', ID_MENU_THEME_DARK },
    { FVIRTKEY | FCONTROL, '4', ID_MENU_FONT_SEGOE },
    { FVIRTKEY | FCONTROL, '5', ID_MENU_FONT_CONSOLAS },
    { FVIRTKEY | FCONTROL, '6', ID_MENU_FONT_COURIER },
    { FVIRTKEY | FCONTROL | FSHIFT, '1', ID_MENU_FONTSIZE_10 },
    { FVIRTKEY | FCONTROL | FSHIFT, '2', ID_MENU_FONTSIZE_12 },
    { FVIRTKEY | FCONTROL | FSHIFT, '3', ID_MENU_FONTSIZE_14 },
    { FVIRTKEY | FCONTROL | FSHIFT, '4', ID_MENU_FONTSIZE_16 },
    { FVIRTKEY | FCONTROL, VK_OEM_MINUS, ID_CMD_FONTSIZE_DEC },

```

```

        { FVIRTKEY | FCONTROL, VK_OEM_PLUS, ID_CMD_FONTSIZE_INC },
        { FVIRTKEY | FALT, VK_LEFT, 100 },
        { FVIRTKEY | FALT, VK_RIGHT, 101 }
    };
    hAccel = CreateAcceleratorTable(acc, sizeof(acc) / sizeof(acc[0]
    ));
}

void UpdateMenuChecks(HWND hwnd) {
    HMENU hBar = GetMenu(hwnd);
    if (!hBar) return;
    HMENU hSettings = GetSubMenu(hBar, 1);
    if (!hSettings) return;
    HMENU hTheme = GetSubMenu(hSettings, 0);
    HMENU hFont = GetSubMenu(hSettings, 1);
    HMENU hFontSize = GetSubMenu(hSettings, 2);

    ThemeChoice theme = Settings::GetTheme();
    UINT idTheme = (theme == ThemeChoice::System) ?
        ID_MENU_THEME_SYSTEM :
            (theme == ThemeChoice::Light ?
                ID_MENU_THEME_LIGHT : ID_MENU_THEME_DARK);
    if (hTheme) CheckMenuRadioItem(hTheme, ID_MENU_THEME_SYSTEM,
        ID_MENU_THEME_DARK, idTheme, MF_BYCOMMAND);

    std::wstring fontFace = Settings::GetFontFace();
    UINT idFont = ID_MENU_FONT_SEGOE;
    if (_wcsicmp(fontFace.c_str(), L"Consolas") == 0) idFont =
        ID_MENU_FONT_CONSOLAS;
    else if (_wcsicmp(fontFace.c_str(), L"Courier New") == 0) idFont
        = ID_MENU_FONT_COURIER;
    if (hFont) CheckMenuRadioItem(hFont, ID_MENU_FONT_SEGOE,
        ID_MENU_FONT_COURIER, idFont, MF_BYCOMMAND);

    int fontSize = Settings::GetFontSize();
    UINT idSize = (fontSize == 10) ? ID_MENU_FONTSIZE_10 : (fontSize
        == 12) ? ID_MENU_FONTSIZE_12 :
            (fontSize == 14) ? ID_MENU_FONTSIZE_14 :
                ID_MENU_FONTSIZE_16;
    if (hFontSize) CheckMenuRadioItem(hFontSize, ID_MENU_FONTSIZE_10
        , ID_MENU_FONTSIZE_16, idSize, MF_BYCOMMAND);
}

}

// Navigation module

#ifdef UNICODE
#define UNICODE
#endif
#ifdef _UNICODE
#define _UNICODE
#endif

#include "../include/ui.h"
#include "../include/filesystem.h"
#include "../include/settings.h"
#include <windows.h>
#include <commctrl.h>
#include <shellapi.h>
#include <algorithm>

```

```

namespace UI {
    void UpdateNavButtons(UIControls &controls, UIState &state) {
        EnableWindow(controls.hBackBtn, state.historyIndex > 0);
        EnableWindow(controls.hFwdBtn, state.historyIndex + 1 < (int)
            state.history.size());
    }

    void NavigateTo(HWND hwnd, UIControls &controls, UIState &state,
        const std::wstring &path, bool addToHistory) {

        if (addToHistory) {
            if (state.historyIndex + 1 < (int)state.history.size()) {
                state.history.erase(state.history.begin() + state.
                    historyIndex + 1, state.history.end());
            }
            if (state.history.empty() || _wcsicmp(state.history.back().
                c_str(), path.c_str()) != 0) {
                state.history.push_back(path);
            }
            state.historyIndex = (int)state.history.size() - 1;
        }

        bool isNetworkPath = (path.length() >= 2 && path[0] == L'\\' &&
            path[1] == L'\\');

        if (isNetworkPath) {
            HTREEITEM root = TreeView_GetRoot(controls.hTreeBottom);
            std::function<HTREEITEM(HTREEITEM, const std::wstring&)>
                searchTree =
                [&](HTREEITEM hItem, const std::wstring& searchPath) ->
                    HTREEITEM {
                    while (hItem) {
                        TVITEMW tvi{}; tvi.mask = TVIF_PARAM; tvi.hItem
                            = hItem;
                        TreeView_GetItem(controls.hTreeBottom, &tvi);
                        Filesystem::NodeData *data = reinterpret_cast<
                            Filesystem::NodeData*>(tvi.lParam);
                        if (data && _wcsicmp(data->path.c_str(),
                            searchPath.c_str()) == 0) {
                            return hItem;
                        }
                    }
                    HTREEITEM child = TreeView_GetChild(controls.
                        hTreeBottom, hItem);
                    if (child) {
                        HTREEITEM found = searchTree(child,
                            searchPath);
                        if (found) return found;
                    }
                    hItem = TreeView_GetNextSibling(controls.
                        hTreeBottom, hItem);
                }
            return nullptr;
        };
        HTREEITEM target = searchTree(root, path);
        if (target) {
            state.suppressTreeSelChange = true;
            TreeView_SelectItem(controls.hTreeBottom, target);
            TreeView_EnsureVisible(controls.hTreeBottom, target);
            state.suppressTreeSelChange = false;
        }
    }
}

```



```

    }
} else {
    HTREEITEM target = Filesystem::EnsurePathInTree(controls.
        hTree, path);
    if (target) {
        state.suppressTreeSelChange = true;
        TreeView_SelectItem(controls.hTree, target);
        TreeView_EnsureVisible(controls.hTree, target);
        state.suppressTreeSelChange = false;
    }
}

state.currentPath = path;
state.listItems.clear();

// Update path edit IMMEDIATELY before loading directory to keep
// it synchronized
if (controls.hPathEdit) {
    SetWindowTextW(controls.hPathEdit, path.c_str());
}

// Ensure list is in report mode with columns set up
HWND hHeader = ListView_GetHeader(controls.hList);
if (!IsReportMode(controls.hList)) {
    SetListViewMode(controls.hList, LVS_REPORT);
    DWORD ex = ListView_GetExtendedListViewStyle(controls.hList)
        ;
    ex |= LVS_EX_FULLROWSELECT | LVS_EX_LABELTIP;
    ThemeChoice effective = Settings::GetTheme();
    if (effective == ThemeChoice::System) {
        effective = Settings::QuerySystemAppsUseLightTheme() ?
            ThemeChoice::Light : ThemeChoice::Dark;
    }
    if (effective != ThemeChoice::Dark) {
        ex |= LVS_EX_GRIDLINES; // Only use system gridlines in
            light theme
    }
    ListView_SetExtendedListViewStyle(controls.hList, ex);
} else {
    // Make sure columns exist - check header
    if (hHeader && Header_GetItemCount(hHeader) == 0) {
        SetListColumns(controls.hList);
    }
    AdjustReportColumns(controls.hList);
}

// Ensure image list is set before populating
SHFILEINFOW sfi{};
HIMAGELIST hSmall = (HIMAGELIST)SHGetFileInfoW(L"C:\\",
    FILE_ATTRIBUTE_DIRECTORY, &sfi, sizeof(sfi),
    SHGFI_SYSICONINDEX | SHGFI_SMALLICON);
if (hSmall) {
    ListView_SetImageList(controls.hList, hSmall, LVSIL_SMALL);
}

Filesystem::PopulateListForDirectory(controls.hList, path, state
    .listItems, controls.hEmptyLabel);

// Show/hide list view and header based on whether we have items
if (!state.listItems.empty()) {

```

```

        ShowWindow(controls.hList, SW_SHOW);
        if (hHeader) ShowWindow(hHeader, SW_SHOW);
        if (controls.hEmptyLabel) {
            ShowWindow(controls.hEmptyLabel, SW_HIDE);
        }
    } else {
        // Hide list and header when showing empty message
        if (hHeader) ShowWindow(hHeader, SW_HIDE);
        ShowWindow(controls.hList, SW_HIDE);
    }

    UpdateNavButtons(controls, state);
    UpdateStatusBar(controls.hStatusBar, state.listItems);

    // Update WindowData with new state
    UI::UpdateWindowData(hwnd, controls, state);
}

}

// Subclass procedures module

#ifndef UNICODE
#define UNICODE
#endif
#ifndef _UNICODE
#define _UNICODE
#endif

#include "../include/ui.h"
#include "../include/settings.h"
#include <windows.h>
#include <commctrl.h>

namespace UI {
    LRESULT CALLBACK ListViewSubclassProc(HWND hwnd, UINT uMsg, WPARAM
        wParam, LPARAM lParam) {
        HWND hParent = GetParent(hwnd);
        if (!hParent) return DefWindowProc(hwnd, uMsg, wParam, lParam);

        ThemeChoice effective = Settings::GetTheme();
        if (effective == ThemeChoice::System) {
            effective = Settings::QuerySystemAppsUseLightTheme() ?
                ThemeChoice::Light : ThemeChoice::Dark;
        }

        if (uMsg == WM_PAINT) {
            HWND hEmptyLabel = GetEmptyLabel(hParent);
            if (IsListViewEmpty(hParent) && hEmptyLabel &&
                IsWindowVisible(hEmptyLabel)) {
                WNDPROC oldProc = GetOldListProc(hParent);
                if (!oldProc) return DefWindowProc(hwnd, uMsg, wParam,
                    lParam);

                // Draw empty message
                LRESULT result = CallWindowProcW(oldProc, hwnd, uMsg,
                    wParam, lParam);
                HDC hdc = GetDC(hwnd);
                if (hdc) {
                    RECT rc; GetClientRect(hwnd, &rc);
                    COLORREF grayColor = RGB(128, 128, 128);

```

```

        if (effective == ThemeChoice::Dark) grayColor =
            RGB(160, 160, 160);
        COLORREF oldColor = SetTextColor(hdc, grayColor);
        int oldBkMode = SetBkMode(hdc, TRANSPARENT);
        HFONT oldFont = nullptr;
        HFONT hFont = Settings::GetFont();
        if (hFont) oldFont = (HFONT)SelectObject(hdc, hFont)
            ;
        wchar_t text[256];
        GetWindowTextW(hEmptyLabel, text, 256);
        if (wcslen(text) == 0) wcscpy(text, L"Здесь пусто");
        SIZE textSize;
        GetTextExtentPoint32W(hdc, text, (int)wcslen(text),
            &textSize);
        int x = (rc.right - rc.left - textSize.cx) / 2;
        int y = (rc.bottom - rc.top - textSize.cy) / 2;
        TextOutW(hdc, x, y, text, (int)wcslen(text));
        SetTextColor(hdc, oldColor);
        SetBkMode(hdc, oldBkMode);
        if (oldFont) SelectObject(hdc, oldFont);
        ReleaseDC(hwnd, hdc);
    }
    return result;
}

WNDPROC oldProc = GetOldListProc(hParent);
if (!oldProc) return DefWindowProc(hwnd, uMsg, wParam, lParam);
return CallWindowProcW(oldProc, hwnd, uMsg, wParam, lParam);
}

// Subclass procedure for border controls to handle dark theme
borders
LRESULT CALLBACK BorderSubclassProc(HWND hwnd, UINT uMsg, WPARAM
wParam, LPARAM lParam, UINT_PTR uIdSubclass, DWORD_PTR dwRefData
) {
    if (uMsg == WM_NCPAINT) {
        ThemeChoice effective = Settings::GetTheme();
        if (effective == ThemeChoice::System) {
            effective = Settings::QuerySystemAppsUseLightTheme() ?
                ThemeChoice::Light : ThemeChoice::Dark;
        }
        if (effective == ThemeChoice::Dark) {
            HDC hdc = GetWindowDC(hwnd);
            if (hdc) {
                RECT rc;
                GetWindowRect(hwnd, &rc);
                OffsetRect(&rc, -rc.left, -rc.top);
                // Draw dark border (gray)
                COLORREF borderColor = RGB(80, 80, 80); // Gray
                border for dark theme
                HBRUSH hBorderBrush = CreateSolidBrush(borderColor);
                FrameRect(hdc, &rc, hBorderBrush);
                DeleteObject(hBorderBrush);
                ReleaseDC(hwnd, hdc);
                return 0;
            }
        }
    }
}

if (uMsg == WM_THEMECHANGED || uMsg == WM_SYSCOLORCHANGE) {

```

```

        // Invalidate to force redraw when theme changes
        InvalidateRect(hwnd, nullptr, FALSE);
    }
    return DefSubclassProc(hwnd, uMsg, wParam, lParam);
}

LRESULT CALLBACK HeaderSubclassProc(HWND hwnd, UINT uMsg, WPARAM
wParam, LPARAM lParam) {
    HWND hParent = GetParent(hwnd);
    if (!hParent) return DefWindowProc(hwnd, uMsg, wParam, lParam);

    ThemeChoice effective = Settings::GetTheme();
    if (effective == ThemeChoice::System) {
        effective = Settings::QuerySystemAppsUseLightTheme() ?
            ThemeChoice::Light : ThemeChoice::Dark;
    }

    COLORREF bgColor = Settings::GetBackgroundColor();
    COLORREF textColor = Settings::GetTextColor();
    HBRUSH hBgBrush = CreateSolidBrush(bgColor);

    if (uMsg == WM_ERASEBKGND) {
        HDC hdc = (HDC)wParam;
        RECT rc; GetClientRect(hwnd, &rc);
        FillRect(hdc, &rc, hBgBrush);
        DeleteObject(hBgBrush);
        return 1;
    }
    if (uMsg == WM_PAINT) {
        PAINTSTRUCT ps;
        HDC hdc = BeginPaint(hwnd, &ps);
        if (hdc) {
            RECT rc; GetClientRect(hwnd, &rc);
            // Fill background
            FillRect(hdc, &rc, hBgBrush);

            if (effective == ThemeChoice::Dark) {
                // Draw header items manually for dark theme
                int itemCount = Header_GetItemCount(hwnd);
                int x = 0;
                for (int i = 0; i < itemCount; i++) {
                    HDITEMW hdi{};
                    wchar_t buf[256] = {0};
                    hdi.mask = HDI_TEXT | HDI_WIDTH | HDI_FORMAT;
                    hdi.pszText = buf;
                    hdi.cchTextMax = 256;
                    if (Header_GetItem(hwnd, i, &hdi)) {
                        RECT itemRect = {x, 0, x + hdi.cxy, rc.
                            bottom};
                        // Draw separator line (gray)
                        if (i < itemCount - 1) {
                            HPEN hPen = CreatePen(PS_SOLID, 1,
                                RGB(60, 60, 60));
                            HPEN hOldPen = (HPEN)SelectObject(hdc,
                                hPen);
                            MoveToEx(hdc, itemRect.right - 1,
                                itemRect.top, nullptr);
                            LineTo(hdc, itemRect.right - 1, itemRect
                                .bottom);
                            SelectObject(hdc, hOldPen);

```

```

        DeleteObject(hPen);
    }

    // Draw text
    SetTextColor(hdc, textColor);
    SetBkColor(hdc, bgColor);
    int oldBkMode = SetBkMode(hdc, TRANSPARENT);
    DrawTextW(hdc, buf, -1, &itemRect, DT_LEFT |
        DT_VCENTER | DT_SINGLELINE);
    SetBkMode(hdc, oldBkMode);

    x += hdi.cxy;
}
}
} else {
    // Use default drawing for light theme
    WNDPROC oldProc = GetOldHeaderProc(hParent);
    if (oldProc) {
        CallWindowProcW(oldProc, hwnd, WM_PAINT, (WPARAM
            )hdc, 0);
    }
}

    EndPaint(hwnd, &ps);
}
DeleteObject(hBgBrush);
return 0;
}
DeleteObject(hBgBrush);
WNDPROC oldProc = GetOldHeaderProc(hParent);
if (!oldProc) return DefWindowProc(hwnd, uMsg, wParam, lParam);
return CallWindowProcW(oldProc, hwnd, uMsg, wParam, lParam);
}
}

// Theme and font application module

#ifndef UNICODE
#define UNICODE
#endif
#ifndef _UNICODE
#define _UNICODE
#endif

#include "../include/ui.h"
#include "../include/settings.h"
#include <windows.h>
#include <commctrl.h>
#include <dwmapi.h>

// Forward declaration
extern LRESULT CALLBACK HeaderSubclassProc(HWND hwnd, UINT uMsg, WPARAM
    wParam, LPARAM lParam);

namespace UI {
    void ApplyThemeToControls(HWND hwnd, UIControls &controls) {
        COLORREF bgColor = Settings::GetBackgroundColor();
        COLORREF textColor = Settings::GetTextColor();

        if (controls.hTree) {

```

```

        TreeView_SetBkColor(controls.hTree, bgColor);
        TreeView_SetTextColor(controls.hTree, textColor);
        InvalidateRect(controls.hTree, nullptr, TRUE);
    }
    if (controls.hTreeBottom) {
        TreeView_SetBkColor(controls.hTreeBottom, bgColor);
        TreeView_SetTextColor(controls.hTreeBottom, textColor);
        InvalidateRect(controls.hTreeBottom, nullptr, TRUE);
    }
    if (controls.hSplitter) {
        InvalidateRect(controls.hSplitter, nullptr, TRUE);
    }
    if (controls.hList) {
        ThemeChoice effective = Settings::GetTheme();
        if (effective == ThemeChoice::System) {
            effective = Settings::QuerySystemAppsUseLightTheme() ?
                ThemeChoice::Light : ThemeChoice::Dark;
        }
        // For dark theme, use CLR_NONE to prevent background from
        // overwriting items
        if (effective == ThemeChoice::Dark) {
            ListView_SetBkColor(controls.hList, CLR_NONE);
            ListView_SetTextBkColor(controls.hList, CLR_NONE);
        } else {
            ListView_SetBkColor(controls.hList, bgColor);
            ListView_SetTextBkColor(controls.hList, bgColor);
        }
        ListView_SetTextColor(controls.hList, textColor);
        if (effective == ThemeChoice::Dark) {
            HWND hHeader = ListView_GetHeader(controls.hList);
            if (hHeader) {
                UI::SetHeaderProc(hwnd, hHeader, HeaderSubclassProc);
                ;
                InvalidateRect(hHeader, nullptr, TRUE);
            }
        }
        InvalidateRect(controls.hList, nullptr, TRUE);
    }
    if (controls.hEmptyLabel) {
        InvalidateRect(controls.hEmptyLabel, nullptr, TRUE);
    }

    ThemeChoice effective = Settings::GetTheme();
    if (effective == ThemeChoice::System) {
        effective = Settings::QuerySystemAppsUseLightTheme() ?
            ThemeChoice::Light : ThemeChoice::Dark;
    }
    if (effective == ThemeChoice::Dark) {
        BOOL darkMode = TRUE;
        DwmSetWindowAttribute(hwnd, DWMWA_USE_IMMERSIVE_DARK_MODE, &
            darkMode, sizeof(darkMode));
        // Invalidate buttons for dark theme
        if (controls.hBackBtn) InvalidateRect(controls.hBackBtn,
            nullptr, TRUE);
        if (controls.hFwdBtn) InvalidateRect(controls.hFwdBtn,
            nullptr, TRUE);
        if (controls.hViewDetailsBtn) InvalidateRect(controls.
            hViewDetailsBtn, nullptr, TRUE);
        if (controls.hViewLargeBtn) InvalidateRect(controls.
            hViewLargeBtn, nullptr, TRUE);
    }

```

```

        if (controls.hViewSmallBtn) InvalidateRect(controls.
            hViewSmallBtn, nullptr, TRUE);
        if (controls.hGoBtn) InvalidateRect(controls.hGoBtn, nullptr
            , TRUE);
        // Redraw menu bar
        DrawMenuBar(hwnd);
    } else {
        BOOL lightMode = FALSE;
        DwmSetWindowAttribute(hwnd, DWMWA_USE_IMMERSIVE_DARK_MODE, &
            lightMode, sizeof(lightMode));
        DrawMenuBar(hwnd);
    }
    if (controls.hPathEdit) {
        InvalidateRect(controls.hPathEdit, nullptr, TRUE);
    }
}

void ApplyFontToControls(HWND hwnd, UIControls &controls) {
    Settings::RecreateFont();
    HFONT hFont = Settings::GetFont();
    if (!hFont) return;

    if (controls.hTree) SendMessageW(controls.hTree, WM_SETFONT,
        (LPARAM)hFont, TRUE);
    if (controls.hTreeBottom) SendMessageW(controls.hTreeBottom,
        WM_SETFONT, (LPARAM)hFont, TRUE);
    if (controls.hList) {
        SendMessageW(controls.hList, WM_SETFONT, (LPARAM)hFont, TRUE
        );
        UpdateListViewRowHeight(controls.hList);
    }
    if (controls.hBackBtn) SendMessageW(controls.hBackBtn,
        WM_SETFONT, (LPARAM)hFont, TRUE);
    if (controls.hFwdBtn) SendMessageW(controls.hFwdBtn, WM_SETFONT,
        (LPARAM)hFont, TRUE);
    if (controls.hEmptyLabel) SendMessageW(controls.hEmptyLabel,
        WM_SETFONT, (LPARAM)hFont, TRUE);
    if (controls.hPathEdit) SendMessageW(controls.hPathEdit,
        WM_SETFONT, (LPARAM)hFont, TRUE);
    if (controls.hGoBtn) SendMessageW(controls.hGoBtn, WM_SETFONT,
        (LPARAM)hFont, TRUE);

    // Set font for menu bar
    HMENU hMenu = GetMenu(hwnd);
    if (hMenu && hFont) {
        // Menu font is handled by system, but we can force redraw
        NONCLIENTMETRICSW ncm = { sizeof(NONCLIENTMETRICSW) };
        if (SystemParametersInfoW(SPI_GETNONCLIENTMETRICS,
            sizeof(ncm), &ncm, 0)) {
            // Update system metrics would require
            SPI_SETNONCLIENTMETRICS which needs admin
            // Instead, force menu redraw
            DrawMenuBar(hwnd);
        }
    }

    // Relayout controls to match new font size
    UIState* state = UI::GetWindowStateForLayout(hwnd);
    if (state) {
        LayoutChildren(hwnd, controls, *state);
    }
}

```

```

    }
}

// Window data management module

#ifndef UNICODE
#define UNICODE
#endif
#ifndef _UNICODE
#define _UNICODE
#endif

#include "../include/ui.h"
#include <windows.h>
#include <mutex>

// Window data structure to store UIControls and UIState
struct WindowData {
    UIControls controls;
    UIState state;
    std::mutex networkScanMutex;
};

// Store window data in GWLP_USERDATA
namespace {
    WindowData* GetWindowDataInternal(HWND hwnd) {
        return reinterpret_cast<WindowData*>(GetWindowLongPtrW(hwnd,
            GWLP_USERDATA));
    }
}

// Export functions for use in other modules
void SetWindowDataInternal(HWND hwnd, WindowData* data) {
    SetWindowLongPtrW(hwnd, GWLP_USERDATA, reinterpret_cast<LONG_PTR>
        (data));
}

// Export GetWindowData for use in other modules
WindowData* GetWindowData(HWND hwnd) {
    return GetWindowDataInternal(hwnd);
}

namespace UI {
    void SetWindowData(HWND hwnd, UIControls *controls, UIState *state)
    {
        WindowData* existing = GetWindowData(hwnd);
        if (existing) {
            if (controls) existing->controls = *controls;
            if (state) existing->state = *state;
        } else {
            WindowData* wndData = new WindowData;
            if (controls) wndData->controls = *controls;
            if (state) wndData->state = *state;
            SetWindowDataInternal(hwnd, wndData);
        }
    }

    UIControls* GetWindowControls(HWND hwnd) {
        WindowData* wndData = GetWindowData(hwnd);
    }
}

```



```

        return wndData ? &wndData->controls : nullptr;
    }

    UIState* GetWindowState(HWND hwnd) {
        WindowData* wndData = GetWindowData(hwnd);
        return wndData ? &wndData->state : nullptr;
    }

    // Helper functions to update WindowData from external modules
    void UpdateWindowControls(HWND hwnd, const UIControls &controls) {
        WindowData* wndData = GetWindowData(hwnd);
        if (wndData) {
            wndData->controls = controls;
        }
    }

    void UpdateWindowState(HWND hwnd, const UIState &state) {
        WindowData* wndData = GetWindowData(hwnd);
        if (wndData) {
            wndData->state = state;
        }
    }

    void UpdateWindowData(HWND hwnd, const UIControls &controls, const
        UIState &state) {
        WindowData* wndData = GetWindowData(hwnd);
        if (wndData) {
            wndData->controls = controls;
            wndData->state = state;
        }
    }

    // Helper function to get state for layout
    UIState* GetWindowStateForLayout(HWND hwnd) {
        WindowData* wndData = GetWindowData(hwnd);
        return wndData ? &wndData->state : nullptr;
    }

    // Helper function to set header proc
    void SetHeaderProc(HWND hwnd, HWND hHeader, WNDPROC proc) {
        WindowData* wndData = GetWindowData(hwnd);
        if (wndData && hHeader && !wndData->state.oldHeaderProc) {
            wndData->state.oldHeaderProc = (WNDPROC)
                SetWindowLongPtrW(hHeader, GWLP_WNDPROC, (LONG_PTR)proc)
                ;
        }
    }

    // Helper functions for subclass procedures
    bool IsListViewEmpty(HWND hwnd) {
        WindowData* wndData = GetWindowData(hwnd);
        if (!wndData) return false;
        return wndData->state.listItems.empty();
    }

    HWND GetEmptyLabel(HWND hwnd) {
        WindowData* wndData = GetWindowData(hwnd);
        return wndData ? wndData->controls.hEmptyLabel : nullptr;
    }

```

```

WNDPROC GetOldListProc(HWND hwnd) {
    WindowData* wndData = GetWindowData(hwnd);
    return wndData ? wndData->state.oldListProc : nullptr;
}

WNDPROC GetOldHeaderProc(HWND hwnd) {
    WindowData* wndData = GetWindowData(hwnd);
    return wndData ? wndData->state.oldHeaderProc : nullptr;
}

// Helper functions for control initialization
WindowData* CreateWindowData() {
    return new WindowData;
}

void InitializeWindowData(WindowData* wndData, const UIControls &
controls, const UIState &state) {
    if (wndData) {
        wndData->controls = controls;
        wndData->state = state;
    }
}

HWND GetToolTip(HWND hwnd) {
    WindowData* wndData = GetWindowData(hwnd);
    return wndData ? wndData->controls.hToolTip : nullptr;
}

void SetToolTip(HWND hwnd, HWND hToolTip) {
    WindowData* wndData = GetWindowData(hwnd);
    if (wndData) {
        wndData->controls.hToolTip = hToolTip;
    }
}

void UpdateScannedDevices(HWND hwnd, const std::vector<std::wstring>
&devices) {
    WindowData* wndData = GetWindowData(hwnd);
    if (wndData) {
        std::lock_guard<std::mutex> lock(wndData->networkScanMutex);
        wndData->state.scannedDevices = devices;
    }
}

std::mutex* GetNetworkScanMutex(HWND hwnd) {
    WindowData* wndData = GetWindowData(hwnd);
    return wndData ? &wndData->networkScanMutex : nullptr;
}

std::vector<std::wstring>* GetScannedDevices(HWND hwnd) {
    WindowData* wndData = GetWindowData(hwnd);
    return wndData ? &wndData->state.scannedDevices : nullptr;
}

// Helper functions for network scan state
HTREEITEM GetLoadingItem(HWND hwnd) {
    WindowData* wndData = GetWindowData(hwnd);
    return wndData ? wndData->state.loadingItem : nullptr;
}

```

```

void SetLoadingItem(HWND hwnd, HTREEITEM item) {
    WindowData* wndData = GetWindowData(hwnd);
    if (wndData) {
        wndData->state.loadingItem = item;
    }
}

bool GetNetworkScanInProgress(HWND hwnd) {
    WindowData* wndData = GetWindowData(hwnd);
    return wndData ? wndData->state.networkScanInProgress : false;
}

void SetNetworkScanInProgress(HWND hwnd, bool inProgress) {
    WindowData* wndData = GetWindowData(hwnd);
    if (wndData) {
        wndData->state.networkScanInProgress = inProgress;
    }
}

std::vector<std::wstring> GetScannedDevicesLocked(HWND hwnd) {
    WindowData* wndData = GetWindowData(hwnd);
    if (!wndData) return std::vector<std::wstring>();
    std::lock_guard<std::mutex> lock(wndData->networkScanMutex);
    return wndData->state.scannedDevices;
}

void ClearScannedDevices(HWND hwnd) {
    WindowData* wndData = GetWindowData(hwnd);
    if (wndData) {
        wndData->state.scannedDevices.clear();
    }
}

void DeleteWindowData(HWND hwnd) {
    WindowData* wndData = GetWindowData(hwnd);
    if (wndData) {
        delete wndData;
        SetWindowDataInternal(hwnd, nullptr);
    }
}
}

```

## Utils

```

#include "../include/utils.h"
#include <shlwapi.h>
#include <algorithm>

namespace Utils {
    std::wstring JoinPath(const std::wstring &a, const std::wstring &b)
    {
        if (a.empty()) return b;
        if (a.back() == L'\\') return a + b;
        return a + L"\" + b;
    }

    std::wstring FileTimeToWString(const FILETIME &ft) {
        SYSTEMTIME stUTC{}, stLocal{};
        FileTimeToSystemTime(&ft, &stUTC);
        SystemTimeToTzSpecificLocalTime(nullptr, &stUTC, &stLocal);
    }
}

```

```

        wchar_t buf[64];
        swprintf(buf, 64, L"%04u-%02u-%02u %02u:%02u",
                 stLocal.wYear, stLocal.wMonth, stLocal.wDay, stLocal.
                 wHour, stLocal.wMinute);
        return buf;
    }
}

```

## Main

```

#ifdef _WIN32
#ifndef UNICODE
#define UNICODE
#endif
#ifndef _UNICODE
#define _UNICODE
#endif

// Winsock must be included before windows.h
#include <winsock2.h>
#include <ws2tcpip.h>
#include <windows.h>
#include <commctrl.h>
#include <shlwapi.h>
#include <gdiplus.h>
#include "include/ui.h"
#include "include/config.h"

#pragma comment(lib, "Comctl32.lib")
#pragma comment(lib, "gdiplus.lib")
#pragma comment(lib, "dwmapi.lib")
#pragma comment(lib, "mpr.lib")
#pragma comment(lib, "iphlpapi.lib")
#pragma comment(lib, "ws2_32.lib")
#pragma comment(lib, "shlwapi.lib")

using namespace Gdiplus;

// Initialize Winsock
struct WinsockInitializer {
    WinsockInitializer() {
        WSADATA wsaData;
        WSAStartup(MAKEWORD(2, 2), &wsaData);
    }
    ~WinsockInitializer() {
        WSACleanup();
    }
} g_winsockInit;

int APIENTRY WinMain(HINSTANCE hInstance, HINSTANCE, LPSTR, int nCmdShow)
{
    // Initialize GDI+
    GdiplusStartupInput gdiplusStartupInput;
    ULONG_PTR gdiplusToken;
    GdiplusStartup(&gdiplusToken, &gdiplusStartupInput, nullptr);

    const wchar_t CLASS_NAME[] = L"FileManagerWin";
    WNDCLASSEXW wc{};
    wc.cbSize = sizeof(WNDCLASSEXW);
    wc.lpfnWndProc = UI::WindowProc;

```

```

wc.hInstance = hInstance;
wc.lpszClassName = CLASS_NAME;
wc.hCursor = LoadCursor(nullptr, IDC_ARROW);
wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);

HICON hIcon = nullptr;
HICON hIconSmall = nullptr;

// Try to load icon from resource first
hIcon = (HICON)LoadImageW(hInstance, MAKEINTRESOURCEW(IDI_MAINICON),
    IMAGE_ICON,
    GetSystemMetrics(SM_CXICON),
    GetSystemMetrics(SM_CYICON), 0);
hIconSmall = (HICON)LoadImageW(hInstance,
    MAKEINTRESOURCEW(IDI_MAINICON), IMAGE_ICON,
    GetSystemMetrics(SM_CXSMICON),
    GetSystemMetrics(SM_CYSMICON),
    0);

if (!hIcon) {
    hIcon = LoadIconW(hInstance, MAKEINTRESOURCEW(IDI_MAINICON));
    hIconSmall = hIcon;
}

if (!hIcon) {
    // Try to load icon from icon.png using GDI+
    wchar_t exePath[MAX_PATH];
    GetModuleFileNameW(hInstance, exePath, MAX_PATH);
    PathRemoveFileSpecW(exePath);
    std::wstring iconPath = std::wstring(exePath) + L"\\icon.png";

    if (GetFileAttributesW(iconPath.c_str()) ==
        INVALID_FILE_ATTRIBUTES) {
        iconPath = L"icon.png";
    }

    if (GetFileAttributesW(iconPath.c_str()) !=
        INVALID_FILE_ATTRIBUTES) {
        hIcon = UI::LoadIconFromPNG(iconPath.c_str(), gdiplusToken);
        hIconSmall = hIcon;
    }
}

if (!hIcon) {
    hIcon = LoadIconW(nullptr, IDI_APPLICATION);
    hIconSmall = hIcon;
}

wc.hIcon = hIcon;
wc.hIconSm = hIconSmall;

if (!RegisterClassExW(&wc)) {
    GdiplusShutdown(gdiplusToken);
    return 1;
}

HWND hwnd = CreateWindowExW(0, CLASS_NAME, L"File Manager",
    WS_OVERLAPPEDWINDOW,
    CW_USEDEFAULT, CW_USEDEFAULT, 1000, 700,
    nullptr, nullptr, hInstance, nullptr);

```

```

    if (!hwnd) {
        GdiplusShutdown(gdiplusToken);
        return 1;
    }

    // Set window icon
    if (hIcon) {
        SendMessageW(hwnd, WM_SETICON, ICON_SMALL, (LPARAM)hIconSmall);
        SendMessageW(hwnd, WM_SETICON, ICON_BIG, (LPARAM)hIcon);
        SetClassLongPtrW(hwnd, GCLP_HICON, (LONG_PTR)hIcon);
        SetClassLongPtrW(hwnd, GCLP_HICONSM, (LONG_PTR)hIconSmall);
    }
    ShowWindow(hwnd, nCmdShow);
    UpdateWindow(hwnd);

    MSG msg{};
    while (GetMessage(&msg, nullptr, 0, 0)) {
        // Get accelerator table from window data (may not be available
        // immediately)
        UIControls* controls = UI::GetWindowControls(hwnd);
        HACCEL hAccel = controls ? controls->hAccel : nullptr;

        if (!hAccel || !TranslateAccelerator(hwnd, hAccel, &msg)) {
            TranslateMessage(&msg);
            DispatchMessage(&msg);
        }
    }

    // Cleanup
    if (hIcon) DestroyIcon(hIcon);
    GdiplusShutdown(gdiplusToken);

    return (int)msg.wParam;
}

#else
// Linux/ncurses version - keep existing code
#include <ncurses.h>
// ... existing ncurses code ...
#endif

```

| Обозначение                |  |  |  |  | Наименование                                                                                                                                               |  |  |  |  | Дополнительные сведения |  |  |  |  |
|----------------------------|--|--|--|--|------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|--|-------------------------|--|--|--|--|
|                            |  |  |  |  | <u>Текстовые документы</u>                                                                                                                                 |  |  |  |  |                         |  |  |  |  |
| БГУИР КП 1–40 01 01 101 ПЗ |  |  |  |  | Пояснительная записка                                                                                                                                      |  |  |  |  | 86 с.                   |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  | <u>Графические документы</u>                                                                                                                               |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
| ГУИР 351001 101 СП         |  |  |  |  | Программное средство «Файловый менеджер с возможностью просмотра файлов на устройствах в одной локальной сети». Схема алгоритма получения корневых дисков. |  |  |  |  | Формат А1               |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |
|                            |  |  |  |  |                                                                                                                                                            |  |  |  |  |                         |  |  |  |  |